

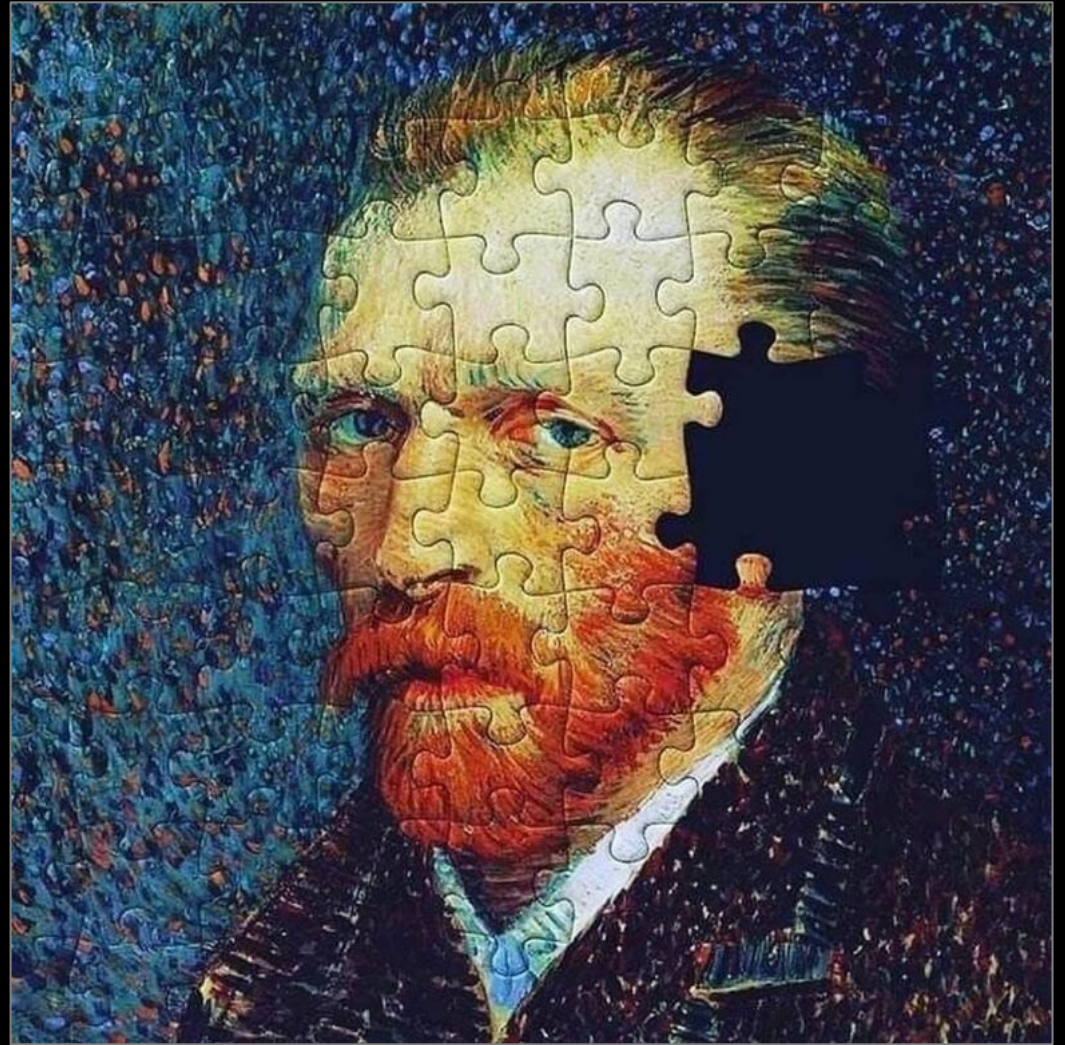
Web Technologies

Web services

microservices

serverless

GraphQL



Prof. Sabin Corneliu Buraga – profs.info.uaic.ro/sabin.buraga/

Assoc. Prof. Andrei Panu – profs.info.uaic.ro/andrei.panu/

„All things were together – then came Mind and set them in order.”

Anaxagoras



alternatives to Web services: microservices



how can the interface of a REST API be described



authorization + authentication based on Web services



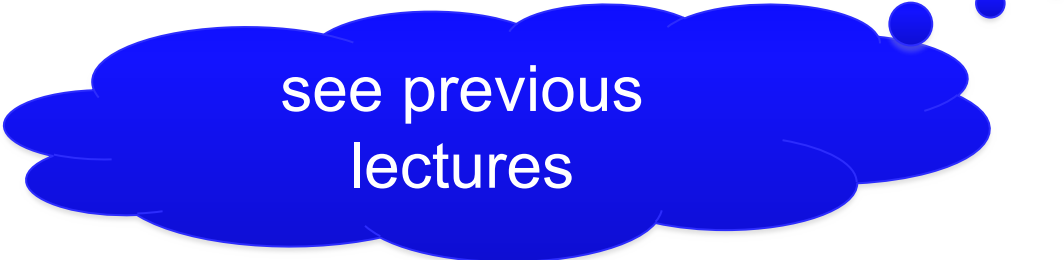
an alternative to REST: GraphQL

Web applications development

Web service

software – remotely used by other applications/services –
providing a specific functionality

its implementation should not be known by the
programmer invoking the service



see previous
lectures



Are there alternatives to Web services?

microservices

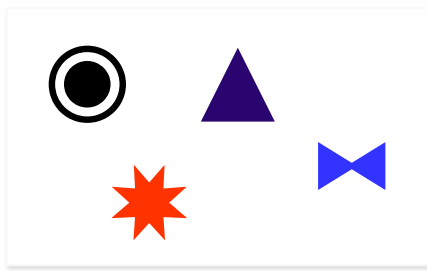
Implement a specific functionality,
available as a single process

self-contained system

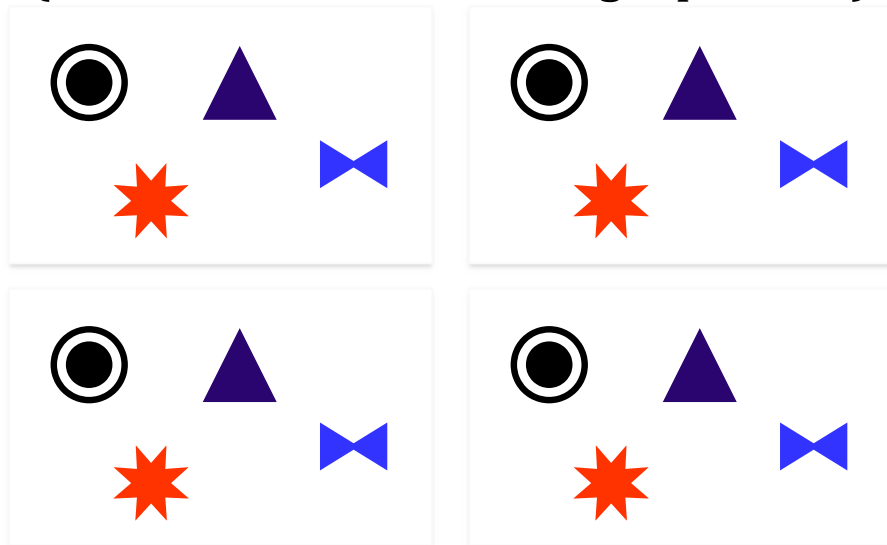
backend component developed to be **replaced**,
not reused

small
each running in its own process
lightweight communication mechanisms (usual, HTTP)
built around business capabilities
independently deployable
minimum of centralized management
may be written in different programming languages
may use different data storage mechanisms

microservice characteristics according to
James Lewis & Martin Fowler, *Microservices* (2014)
martinfowler.com/articles/microservices.html



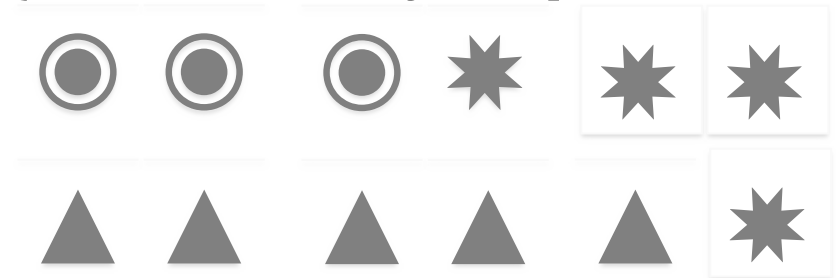
monolithic application
(N functionalities in a single process)



scalability by replicating
the monolith on S servers



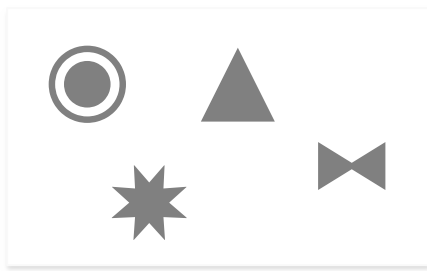
microservices
(each functionality $\equiv$ separate service)



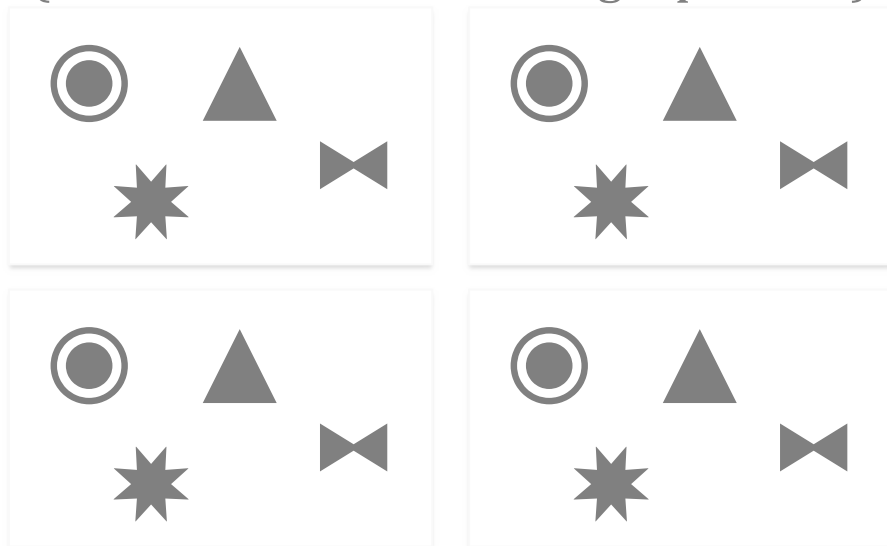
scalability by distributing
services between servers,
replicating them as needed

needs:

modularity, decentralization, and continuous evolution



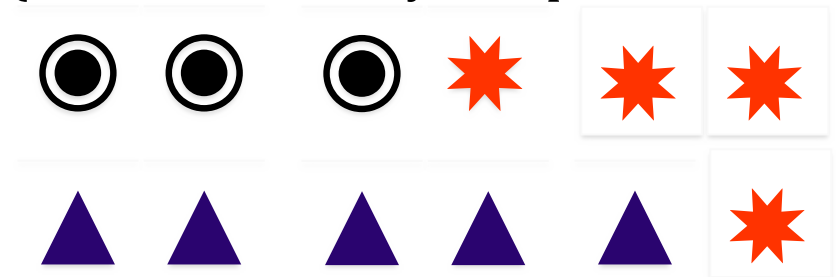
monolithic application
(N functionalities in a single process)



scalability by replicating
the monolith on S servers



microservices
(each functionality $\equiv$ separate service)



scalability by distributing
services between servers,
replicating them as needed

examples of good practice + other resources: microservices.io
Microservices @ The New Stack (2026)
thenewstack.io/microservices/

Benefits of microservices:

isolation
autonomy
individual scalability
resilience
speed
experimentation
rapid feedback
flexibility
replaceability
ecosystem

S. Tilkov, A Question of Size – Modularization & Microservices,
Java Forum Nord 2017:

speakerdeck.com/stilkov/a-question-of-size-modularization-and-microservices

microservices

Functional services

implement specific functionalities
(business operations)

microservices

Functional services

implement specific functionalities
(business operations)

exposed to the service consumer

independent (no side effects)

typically, not shareable

microservices

Control – Infrastructure services

implement non-functional tasks:
authentication, authorization, logging, monitoring,...

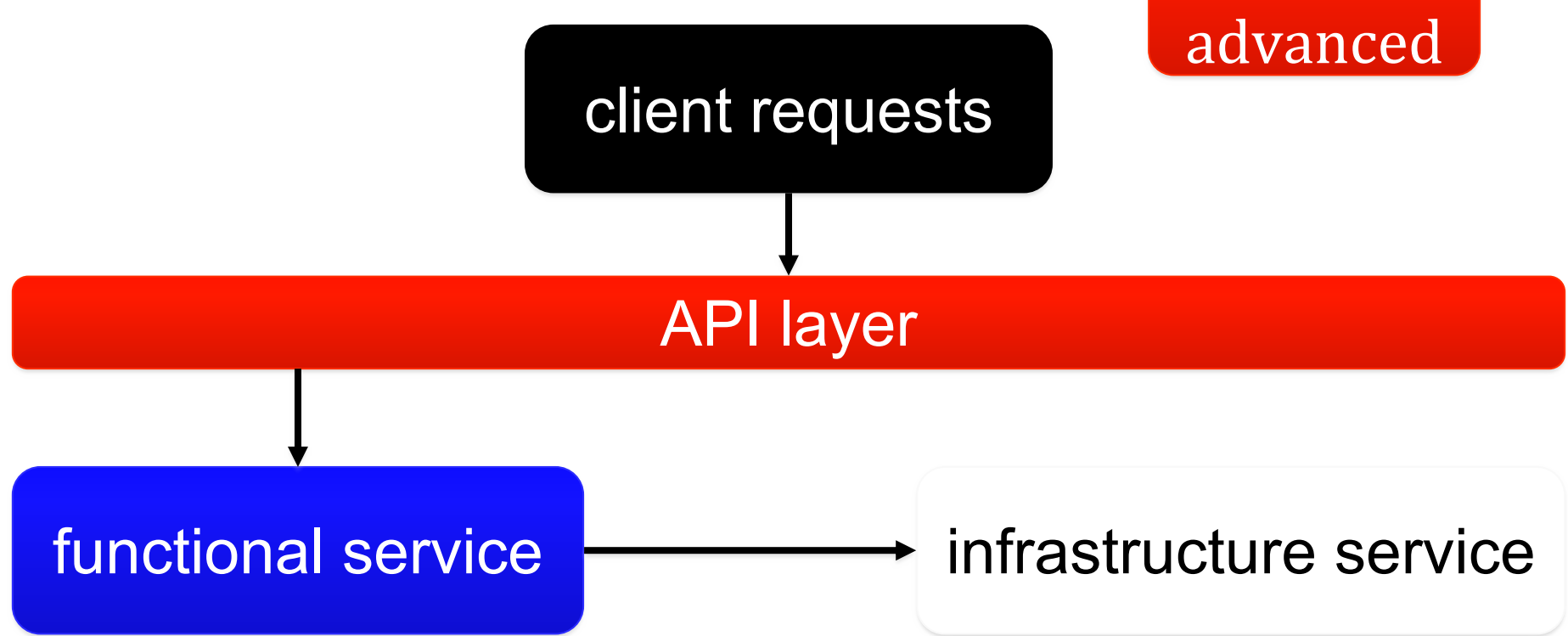
microservices

Control – Infrastructure services

implement non-functional tasks:
authentication, authorization, logging, monitoring,...

not exposed in the exterior – private

could be shared at the application or internal services level



from client requests to API response via functional
microservices based on the ones regarding the
infrastructure

microservices

Aspect of interest:

sharing functionalities

share-as-much-as possible (classical SOA)

versus

share-as-little-as possible (microservices)

microservices

Aspect of interest:

(micro-)services communication – usually, asynchronous

approaches:

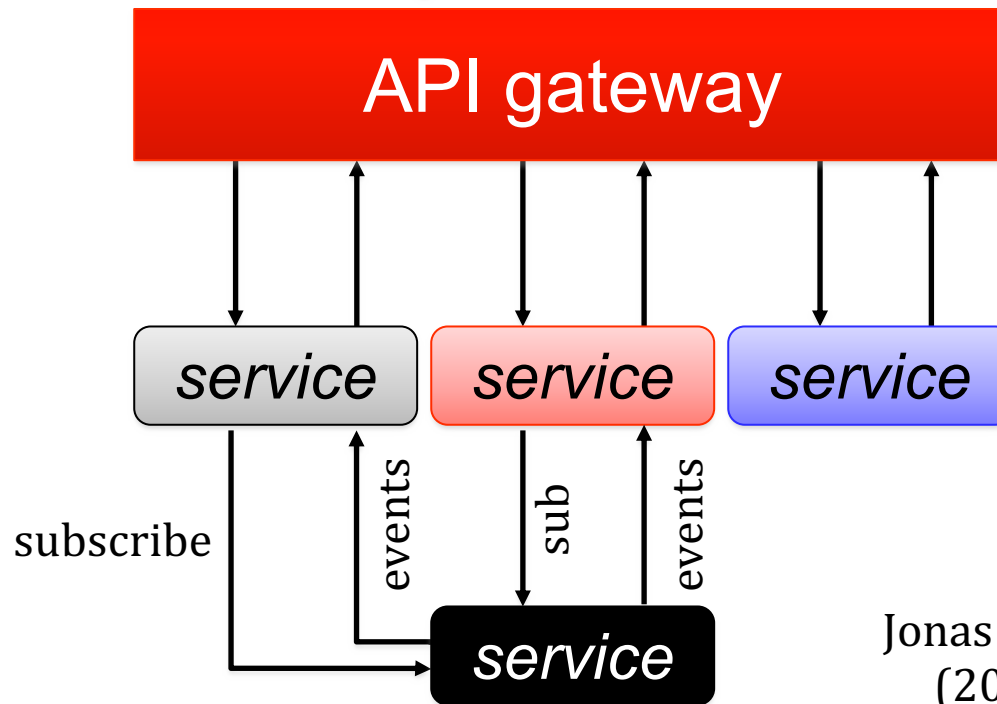
point-to-point

or

publish-subscribe

advanced

simplifying client
access via API



Jonas Bonér
(2016)

internally, (micro-)services can communicate using
publish-subscribe – see the W3C specifications
ActivityPub (2018), **WebSub** (2018)

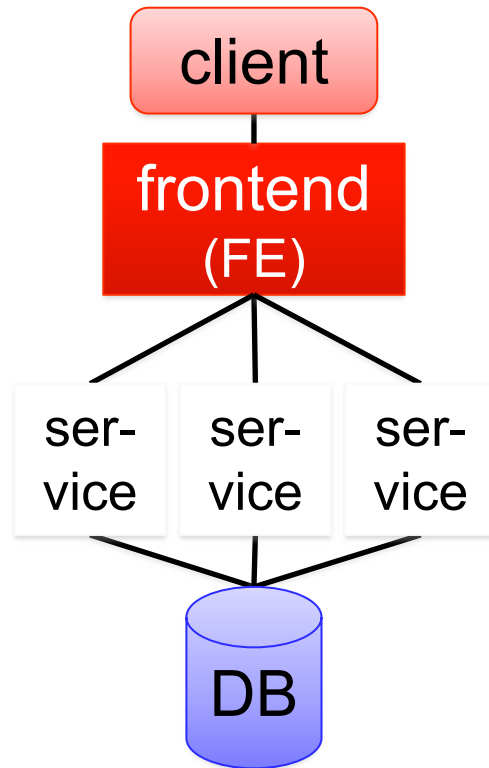
microservices

Commonly, microservice architectures do not include middleware components and do not support abstract interactions between service providers and consumers (contract decoupling)

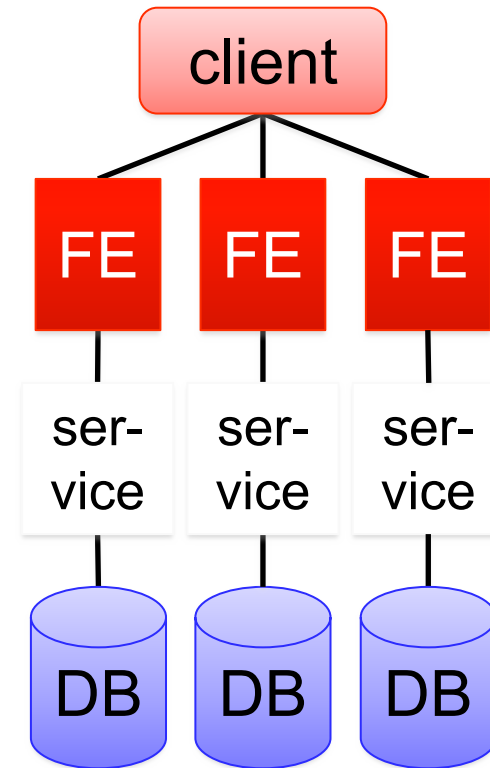
μSOA – Microservice Oriented Architecture

Sam Newman, *Building Microservices (2nd Edition)*, O'Reilly (2021)

advanced



Web service-based
architecture



microservice
architecture

Z. Dehghani, *How to break a Monolith into Microservices* (2018)
martinfowler.com/articles/break-monolith-into-microservices.html

microservices: development

Platforms:

Dapr, Express Serverless, M3O, OpenWhisk, Triton,...

Frameworks:

Akka, Finagle, Eclipse MicroProfile, Micronaut, Moleculer, ScaleCube, Spin, Vert.X, Wangle, etc.

Multi-language SDKs:

CoAP, gRPC, Hprose

Specialized programming languages:

Ballerina, Jolie, Wing

other software tools listed at

github.com/mfornos/awesome-microservices

API

Access to a (micro-)service usually takes place based on an application programming interface
API (Application Programming Interface)

“any **well-defined interface** that defines the service that one component, module, or application provides to other software elements”
(de Souza et al., 2004)

Web API

Designed and invoked
via current Web technologies - URI, HTTP, JSON, etc.

can be developed according to an architectural style
e.g., REST (REpresentational State Transfer)



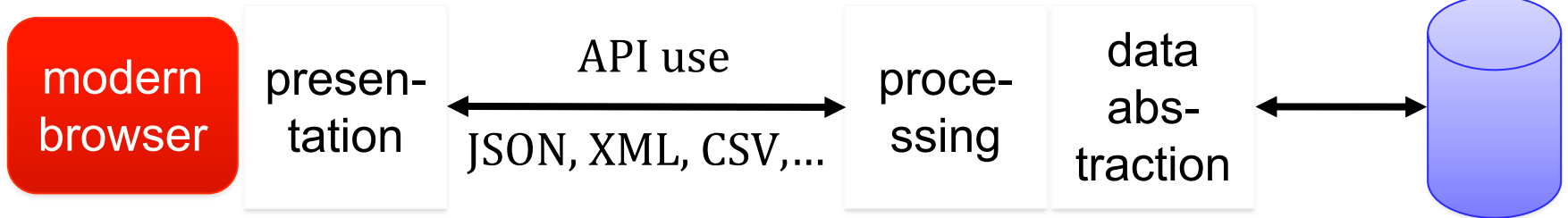
see the previous lecture

public API (available under a license)
versus private API (for internal use)

API: client approach – JavaScript

modern Web client
(HTML5 APIs)

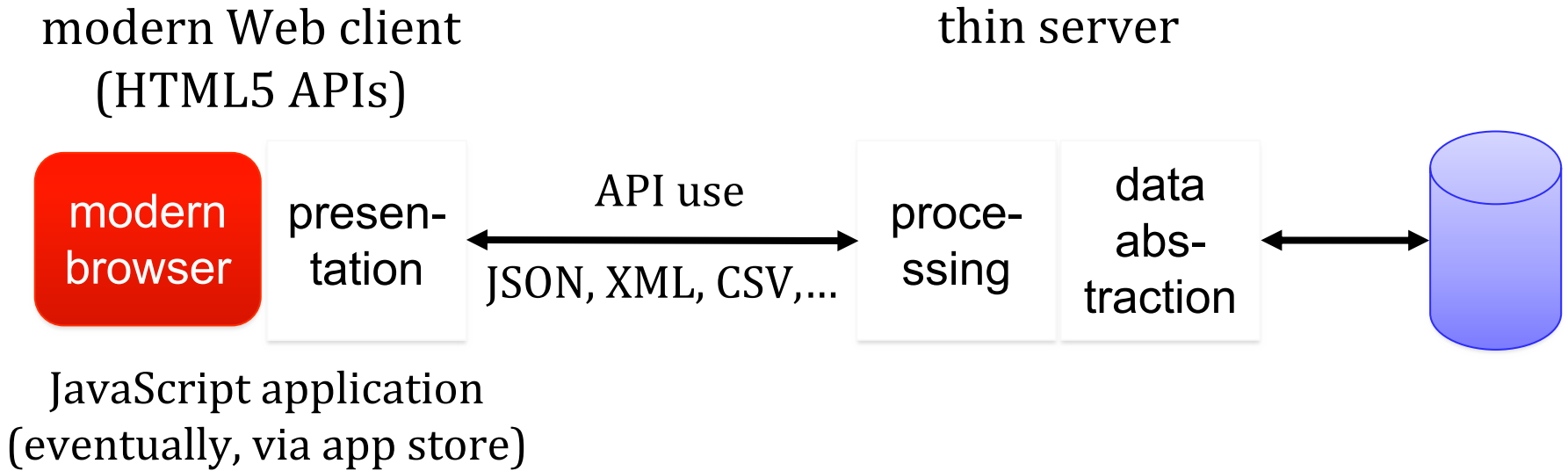
thin server



JavaScript application
(eventually, via app store)

Web browser on
conventional computers,
mobile devices, and others

API: client approach – JavaScript



JavaScript application implementation can use libraries, frameworks, specific components e.g., Angular, htmx, Lit, React, Riot.js, Svelte, Vue

API: client approach – JavaScript

At the client level (front-end),
the creation of user interfaces can be facilitated by:

design systems

micro-frontends

design systems

*“A **design system** is a complete set of standards intended to manage design at scale using reusable components and patterns”*

Therese Fessenden (2021)

www.nngroup.com/articles/design-systems-101/

design systems

Examples:

designsystemsrepo.com/design-systems/

real cases:

Atlassian – atlassian.design

US Web Design System – designsystem.digital.gov

W3C – design-system.w3.org

micro-frontends

“An **architectural style** where independently deliverable frontend applications are composed into a greater whole”

Cam Jackson (2019)

martinfowler.com/articles/micro-frontends.html

micro-frontends

Applying microservices principles to the design and implementation of the user interface components

important benefits:

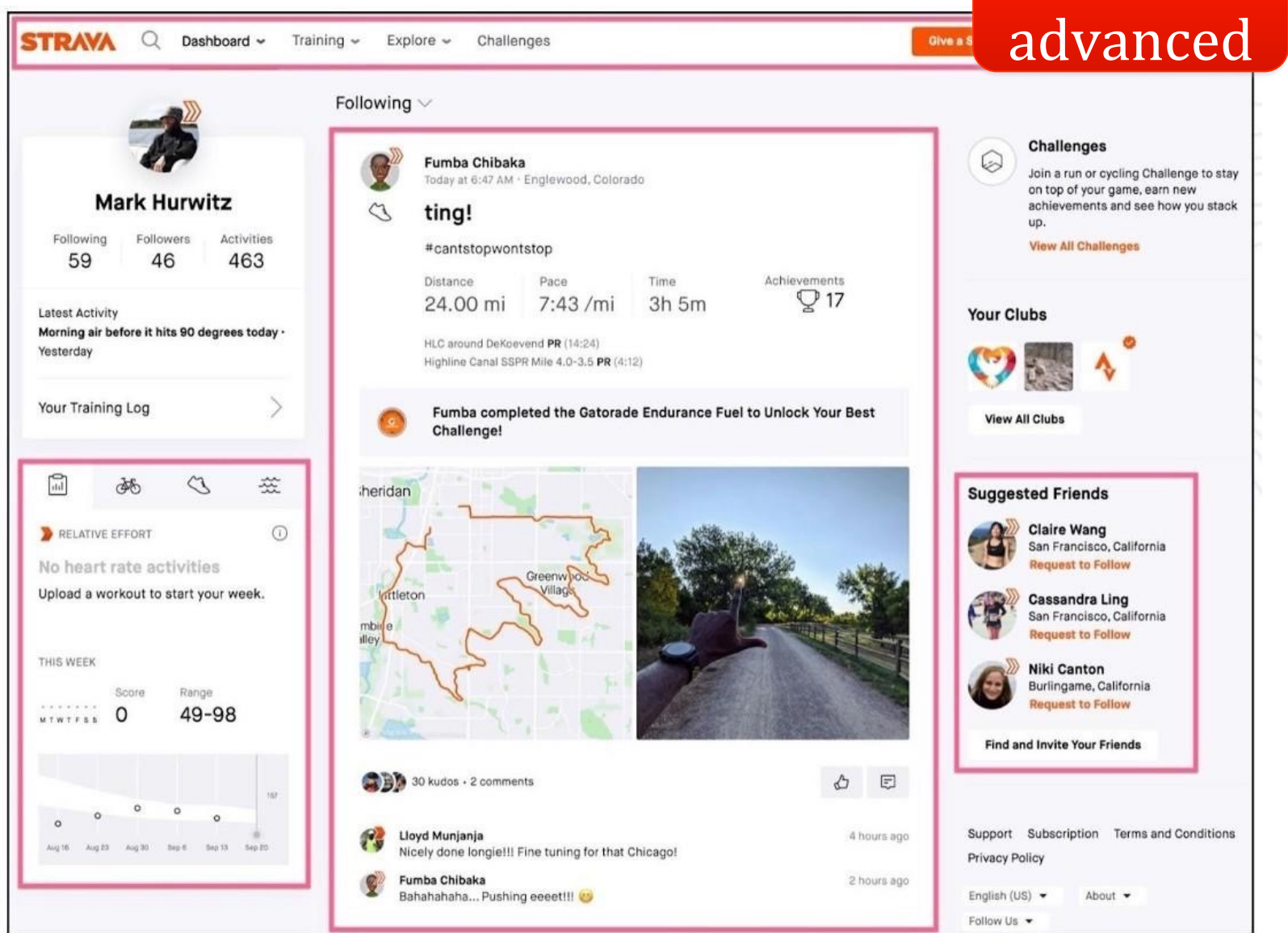
incremental updates

code simplification

UI components developed and exploited independently

autonomous development teams

rapid testing



real case: **Strava** (Mark Hurwitz, 2022)

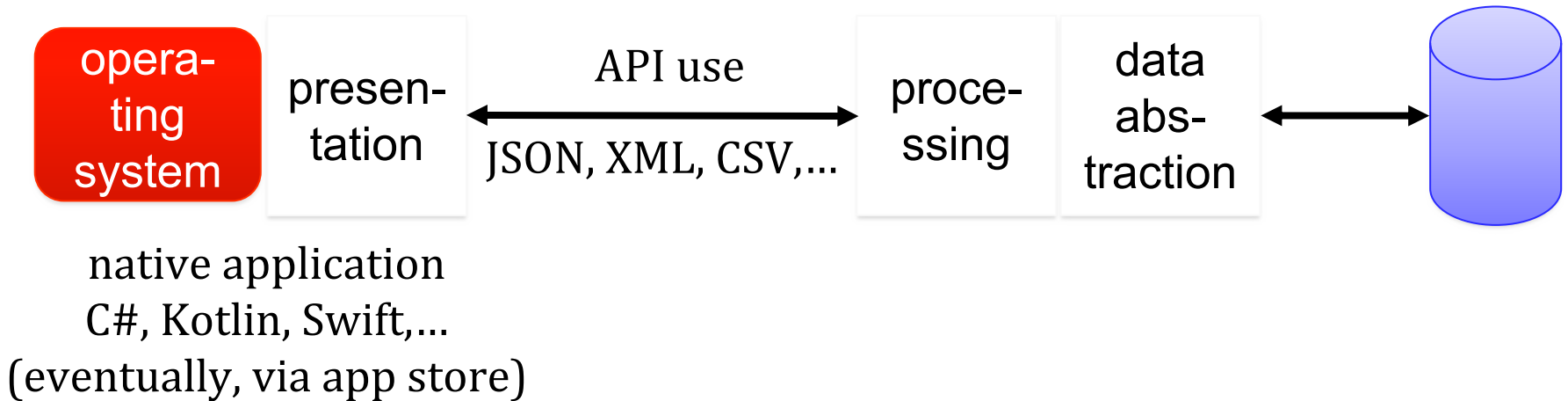
main dashboard page decomposed into different micro-frontends

medium.com/strava-engineering/strava-web-microfrontends-79e9586c0222

API: native applications

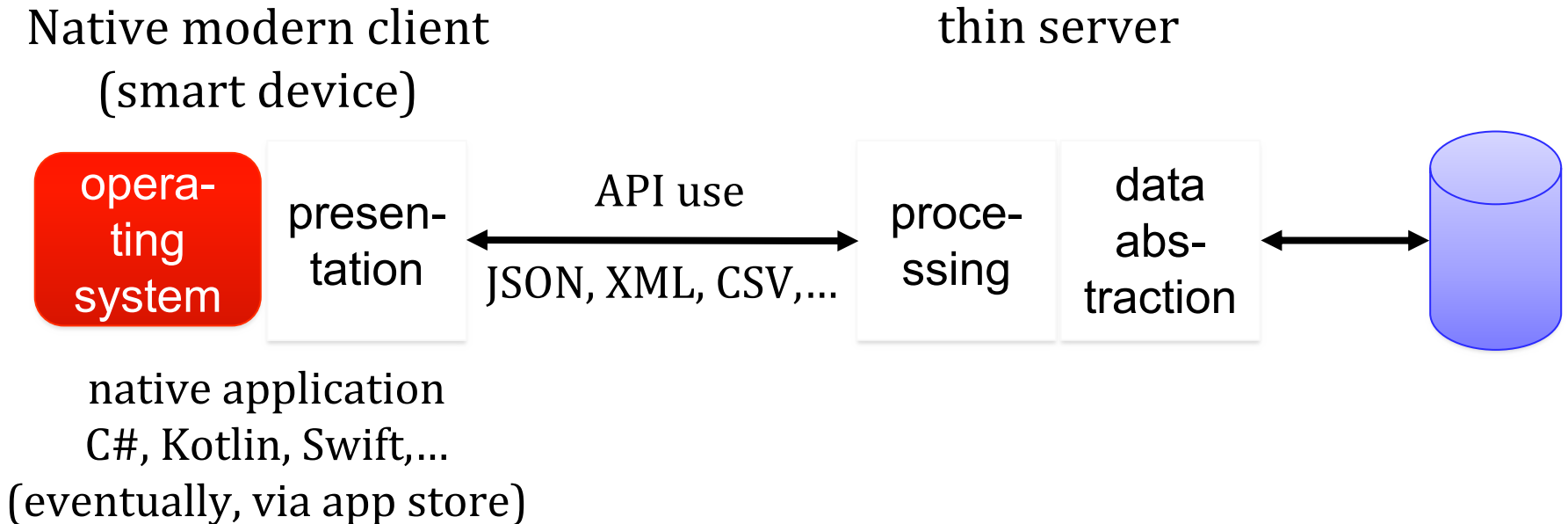
Native modern client
(smart device)

thin server

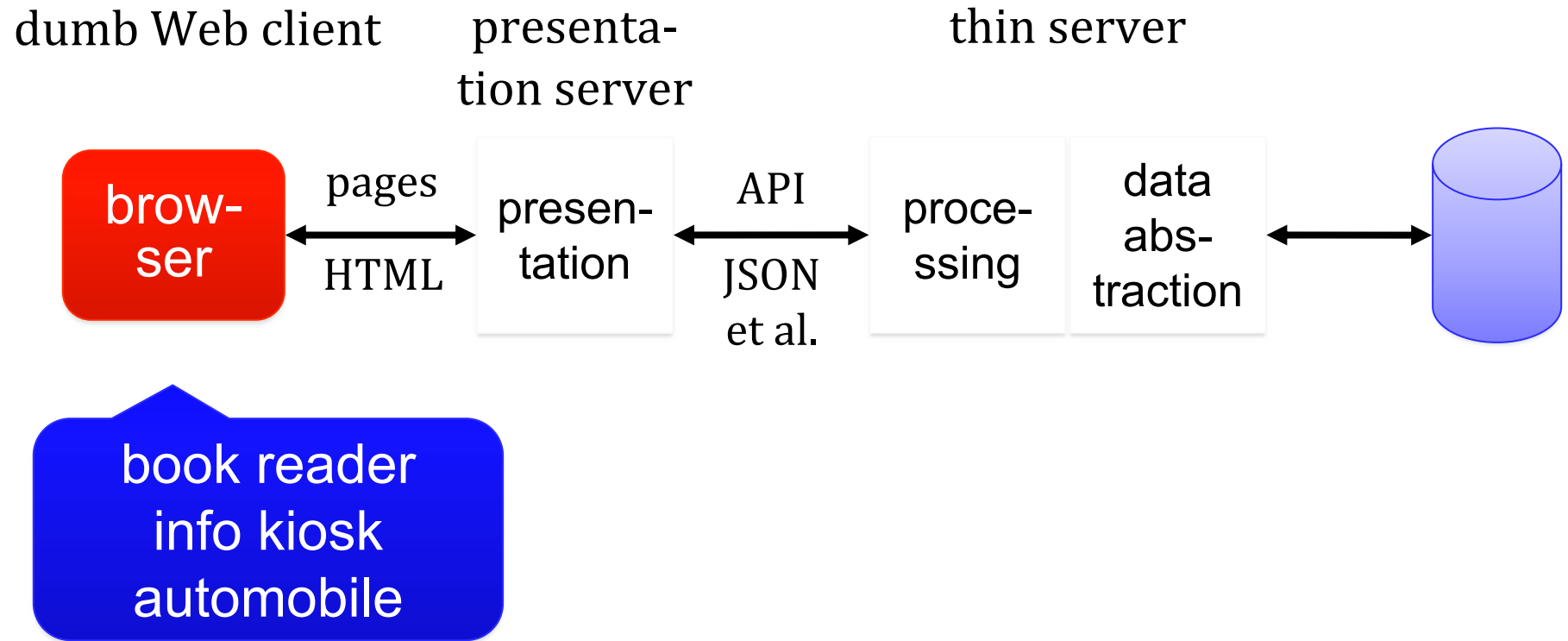


desktop and/or mobile,
smart TV, home appliance,
ambiental device

API: native applications



API: approach based on intermediaries



API: testing tools

curl tool + **LibcURL** library – curl.haxx.se

HTTPIe (command line client) – httpie.io

Hoppscotch – hoppscotch.io

Insomnia – insomnia.rest

Postman – www.postman.com

REST Test Test – resttesttest.com

API: in serverless context

The application depends significantly on external components, available in cloud

(micro-)services exposed via API

vs.

serverless approach

API: in serverless context

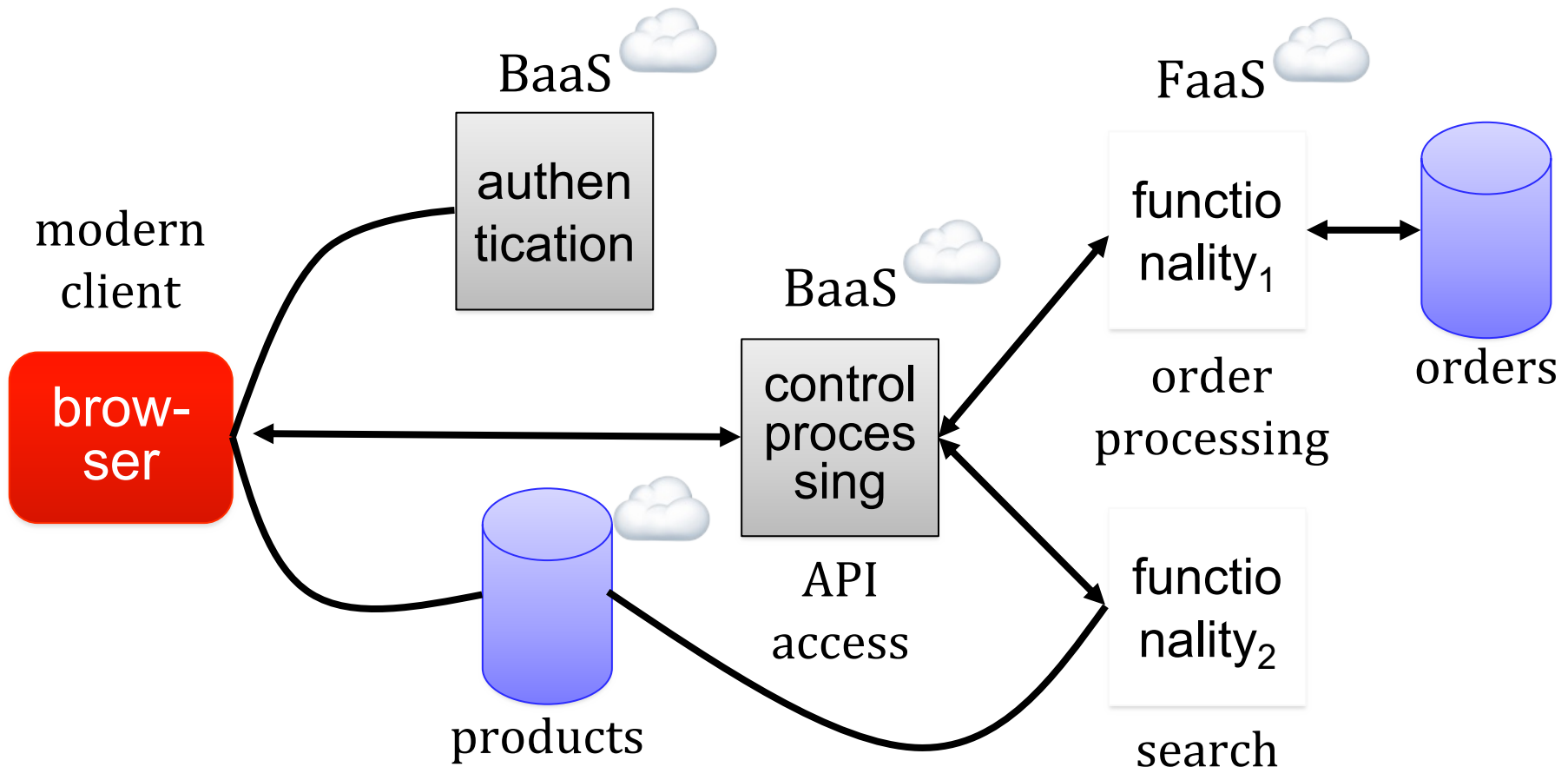
Serverless

layer of abstraction of access to resources
of a cloud platform

Mike Roberts (2018)

martinfowler.com/articles/serverless.html

API: in serverless context



BaaS = (Mobile) Backend As A Service

FaaS = Functions As A Service

API: in serverless context

FaaS – Functions As A Service

cloud functions implementing features exposed to the
service consumer

as small as possible

usually, implementations below 100 lines of code

API: in serverless context

FaaS – Functions As A Service

executed – on the server – **independently** and **asynchronously**, without causing **side effects**

triggered by **events**

the user is not concerned with resource management
and other tasks

API: in serverless context

BaaS – Backend As A Service

Encapsulates infrastructure services which implement
non-functional activities
(authentication, authorization, logging, monitoring, etc.)

private – are not exposed to the exterior

can be shared by internal (micro-)services

API: in serverless context

Serverless computing = FaaS + BaaS

documentation, resources, solutions:
github.com/anaibol/awesome-serverless



How can the interface of a REST API
be described?

API: **abstract description**

OpenAPI Specification – openapis.org

RAML (RESTful API Modeling Language) – raml.org

API Blueprint – apiblueprint.org

other resources of interest:

github.com/Kikobeats/awesome-api

API: abstract description

OpenAPI Specification

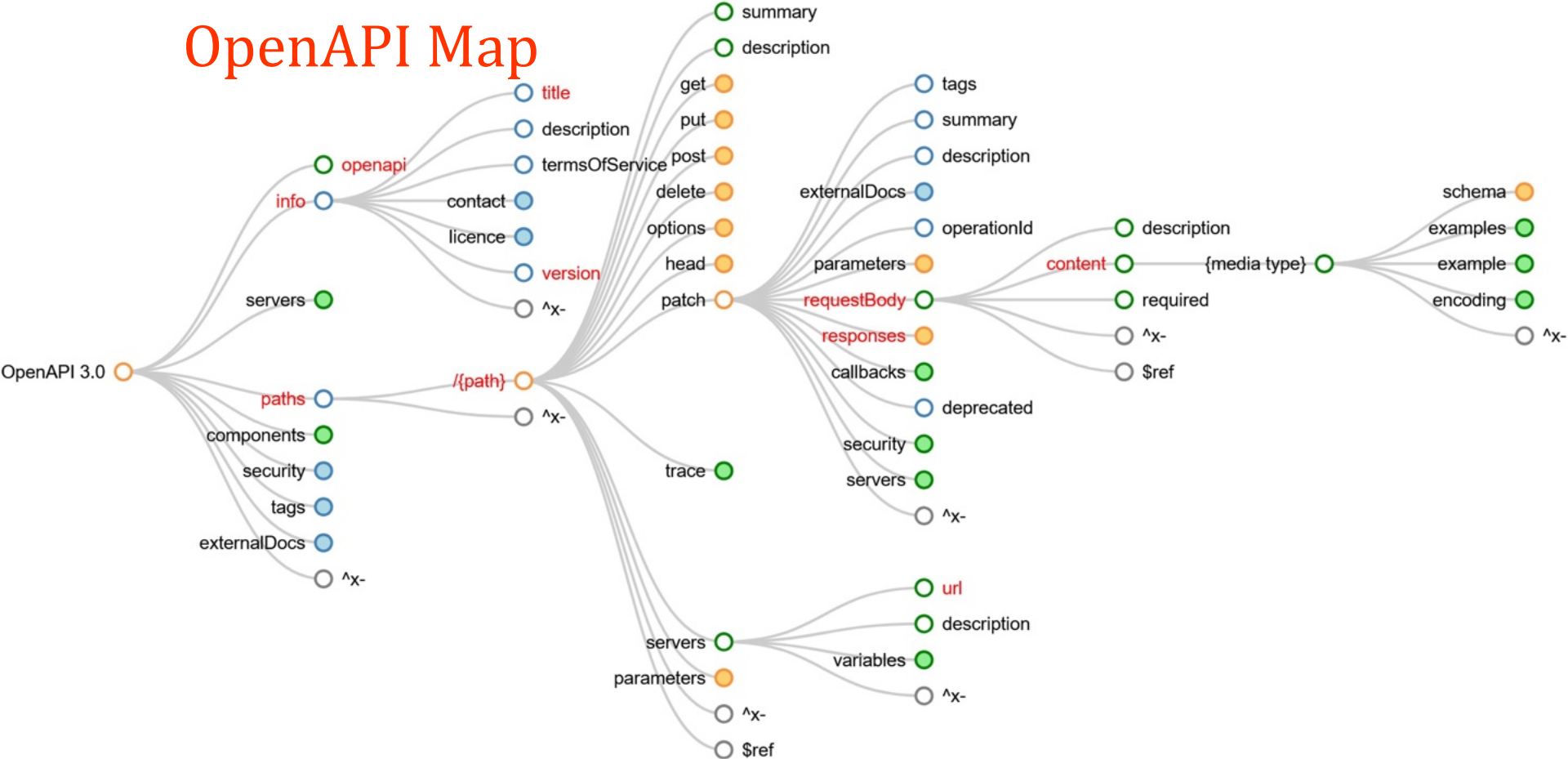
modern solution to declare – platform-independent –
the public interface of a REST API

current version: OpenAPI 3.2.0 (September 2025)

spec.openapis.org/oas/latest.html

used formats: JSON and/or
YAML (YAML Ain't Markup Language)

OpenAPI Map



operationId property



string

Unique string used to identify the operation. The id MUST be unique among all operations described in the API. Tools and libraries MAY use the operationId to uniquely identify an operation, therefore, it is recommended to follow common programming naming conventions.

visual exploration of OpenAPI Specification constructs

openapi-map.apihandyman.io/?version=3.0

```

{ // specifying the access to a Web resource: GET /resource?param
  "openapi": "3.0.0",
  "paths": { // paths (URLs) to the resources offered by the REST API
    "/resource": { // access to the representation of the resource designated by /resource
      "get": { // HTTP method: GET, HEAD, POST, PUT, PATCH, DELETE
        "operationId": "getResource", // specifies the unique name of the desired operation
        "parameters": [ { // the list of parameters
          "name": "param",
          "in": "query", // parameter added to the URL (in query string)
          "schema": { "type" : "string" }
        } ],
        "responses": { // provided answers (success, error,...)
          "200": { // HTTP status code: 200, 400, 403, 405, 500, etc.
            "description": "Success",
            "schema": { // describes (with JSON Schema) the provided answer
              "$ref": "#/definitions/Response"
            }
          }
        }
      }
    }
  }
}

```

the „skeleton” of an OpenAPI document specifying a REST API

```
77 petstore_auth:
78   - write: $ref: string
79   - read: required:
80     post:
81       - name
82       tags:
83         - pet
84       summary: Add a
85       description: A
86       operationId: a
87       requestBody:
88         description:
89         content:
90           application
91           schema:
92             $ref: '#/components/schemas/Pet'
93           application/xml:
94             schema:
95             $ref: '#/components/schemas/Pet'
96           application/x-www-form-urlencoded:
97             schema:
98             $ref: '#/components/schemas/Pet'
99             required: true
100       responses:
101         '200':
102           description: Successful operation
103           content:
104             application/json:
105               schema:
106                 $ref: '#/components/schemas/Pet'
107             application/xml:
108               schema:
109                 $ref: '#/components/schemas/Pet'
110         '405':
111           description: Invalid input
112       security:
113         - petstore_auth:
114           - write:pets
115           - read:pets
```

pet Everything about your Pets [Find out more](#)

PUT /pet Update an existing pet

POST /pet Add a new pet to the store

GET /pet/findByStatus Finds Pets by status

GET /pet/findByTags Finds Pets by tags

GET /pet/{petId} Find pet by ID

POST /pet/{petId} Updates a pet in the store with form data

DELETE /pet/{petId} Deletes a pet

POST /pet/{petId}/uploadImage uploads an image

store Access to Petstore orders [Find out more about our store](#)

GET /store/inventory Returns pet inventories by status

POST /store/order Place an order for a pet

GET /store/order/{orderId} Find purchase order by ID

Dashboard

APIs

Settings

APIs > Bitbucket > Editor

Bitbucket

Search everything...

/teams/{username}

/teams/{username}/followers

/teams/{username}/following

/teams/{username}/hooks

/teams/{username}/hooks/{uid}

/teams/{username}/members

/teams/{username}/pipelines_config/...

/teams/{username}/pipelines_config/...

/teams/{username}/repositories

/user

/user/emails

/user/emails/{email}

/users/{username}

/users/{username}/followers

/users/{username}/following

/users/{username}/hooks

/users/{username}/hooks/{uid}

/users/{username}/pipelines_config/v...

/users/{username}/pipelines_config/v...

/users/{username}/repositories

▼ Data Types (88)

</> account

</> author

</> base_commit

</> branchrestriction

</> commit

</> commitstatus

/teams/{username}/hooks

Design

Source

▼ PATH PARAMETERS (1)

username

> No description.

> string

> QUERY PARAMETERS

> HEADER PARAMETERS

▼ OPERATIONS (2)

Get

Put

Post

Delete

Options

Head

Patch

⋮

> INFO

> PATH PARAMETERS (1)

> QUERY PARAMETERS

> HEADER PARAMETERS

> REQUEST BODY

▼ RESPONSES (3)

201 Created

403 Forbidden

404 Not Found

⋮

inspecting the specification of a public API (here, Bitbucket)

Get Started

- All Tools
- All Categories
- Legacy Tools
- Contributing
- Sponsors
- Sponsor Badges

Collections

- Arazzo Support
- OpenAPI v3.2 Early Adopters
- **Open Source Tools**
- Overlays Support
- SaaS Tools

OpenAPI Tool Categories

- Annotations
- Code generators
- Converters
- Data Validators
- Documentation

Open Source Tools

Free and open source tools for working with OpenAPI descriptions.

These tools have publicly available source code that you can inspect, modify, and contribute to. They're great for self-hosting, customization, and learning how OpenAPI tooling works under the hood.

Languages 5

Platforms 5

PHP ×

CLI ×

Web ×

SaaS ×

Any ×

Clear all

Showing 87 of 160 tools

Search platforms...

- ☒ CLI 42
- ☐ Nodejs 26
- ☒ SaaS 23
- ☒ Web 10
- ☐ Terraform 8
- ☐ Docker 8
- ☐ Unity 6
- ☐ YAML 2

Annotations (1 tool)

Use annotations in your

Name
Swagger PHP Generate i doctrine annotations.

Code generators (6 tools)

Tools to generate code from your OpenAPI Spec, or to generate an OpenAPI Spec from your code.

OpenAPI instruments for conversion, validation, documentation, code generation, testing, etc.: openapi.tools

Filter APIs 

image

[EC2 Image Builder](#)

EC2 Image Builder is a fully managed AWS service that makes it easier to automate the creation,

OpenAPI:

Preferred Version - 2019-12-02

[JSON](#)[YAML](#)[Orig](#)[Docs](#)

► Tools

[Compute Admin Client](#)

OpenAPI:

Preferred Version - 2015-12-01-preview

[JSON](#)[YAML](#)[Orig](#)[Docs](#)

► Tools

[VirtualMachineImageT...](#)

Virtual Machine Image Template

OpenAPI:

Preferred Version - 2019-05-01-preview

[JSON](#)[YAML](#)[Orig](#)[Docs](#)

► All Versions

[Getty Images](#)

Getty Images API

OpenAPI:

Preferred Version - 3

[JSON](#)[YAML](#)[Orig](#)[Docs](#)

► Tools

a list of public APIs accompanied by OpenAPI specifications in JSON and YAML formats: [apis.guru](#)



Charts, simple as a URL. A safe and fast replacement for Google Image Charts



The vehicle images API offers access to original Mercedes-Benz vehicle images. It provides access to exterior



The Bing Custom Image Search API lets you send an image search query to Bing and get back image search



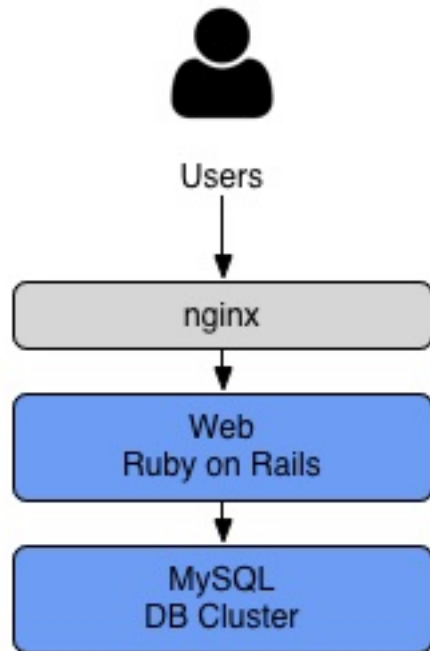
The Image Search API lets you send a search query to Bing and get back a list of relevant images. This section

Case study: **Groupon**

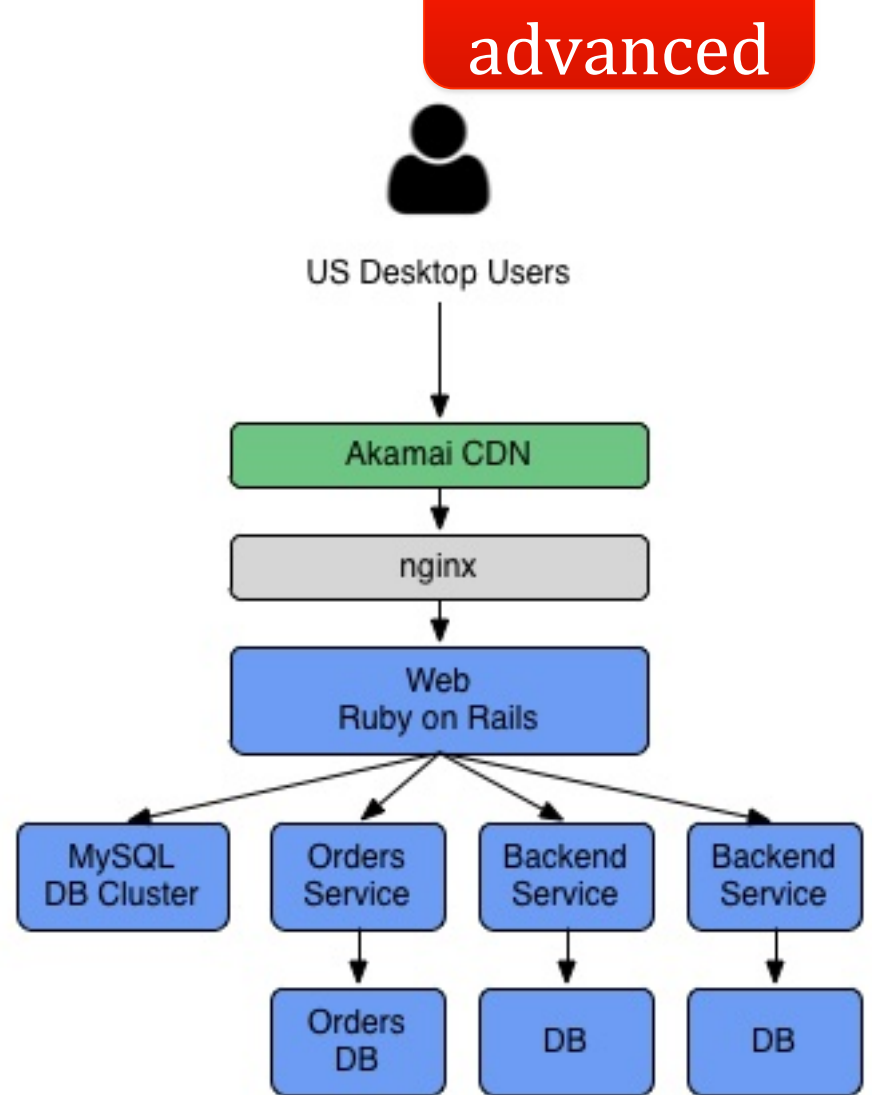
purpose: the presentation of offers of consumer goods

from a monolithic architecture
to one adopting (micro-)services/REST APIs

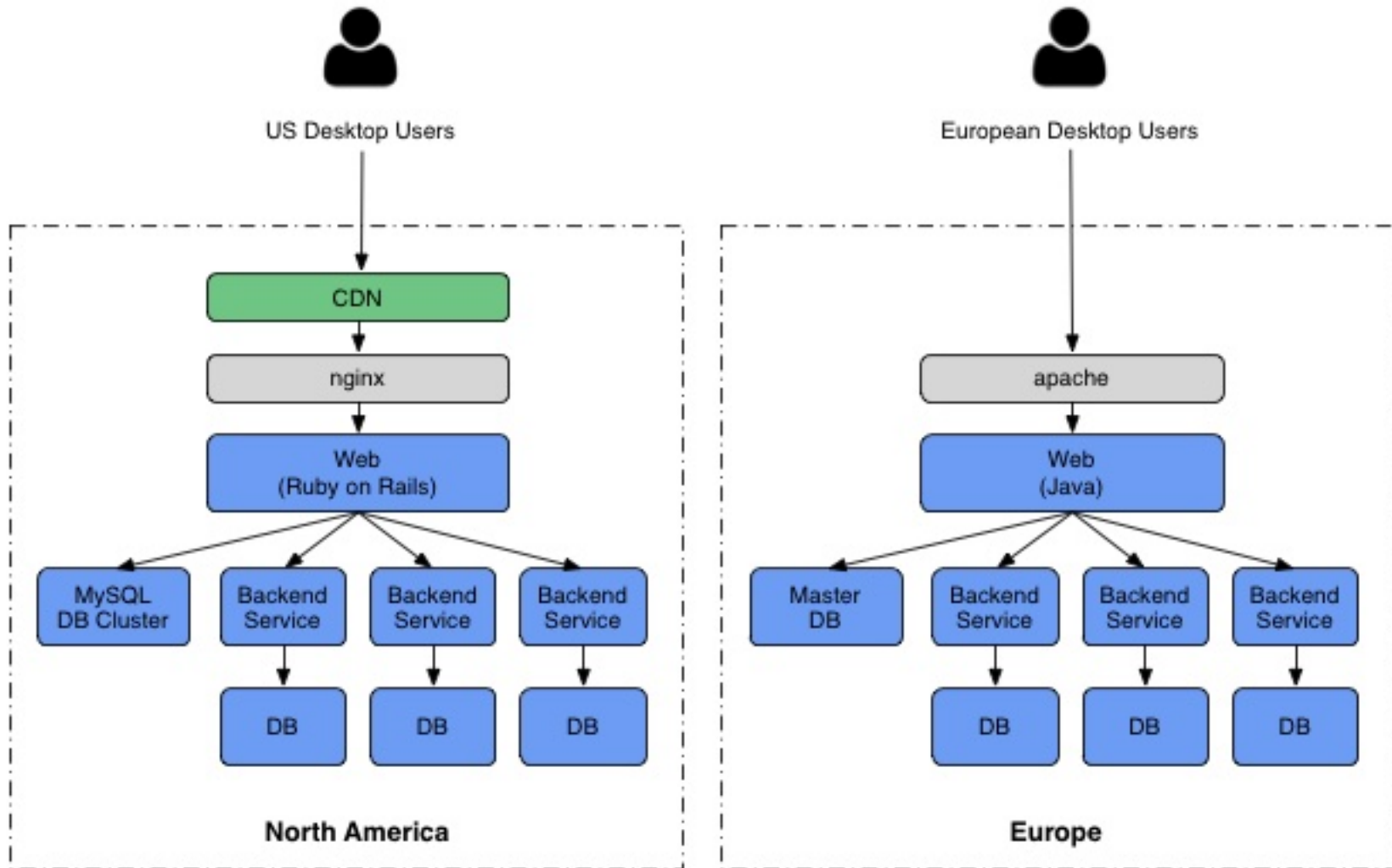
engineering.groupon.com/2013/misc/i-tier-dismantling-the-monoliths/



initial architecture
traditional MVC

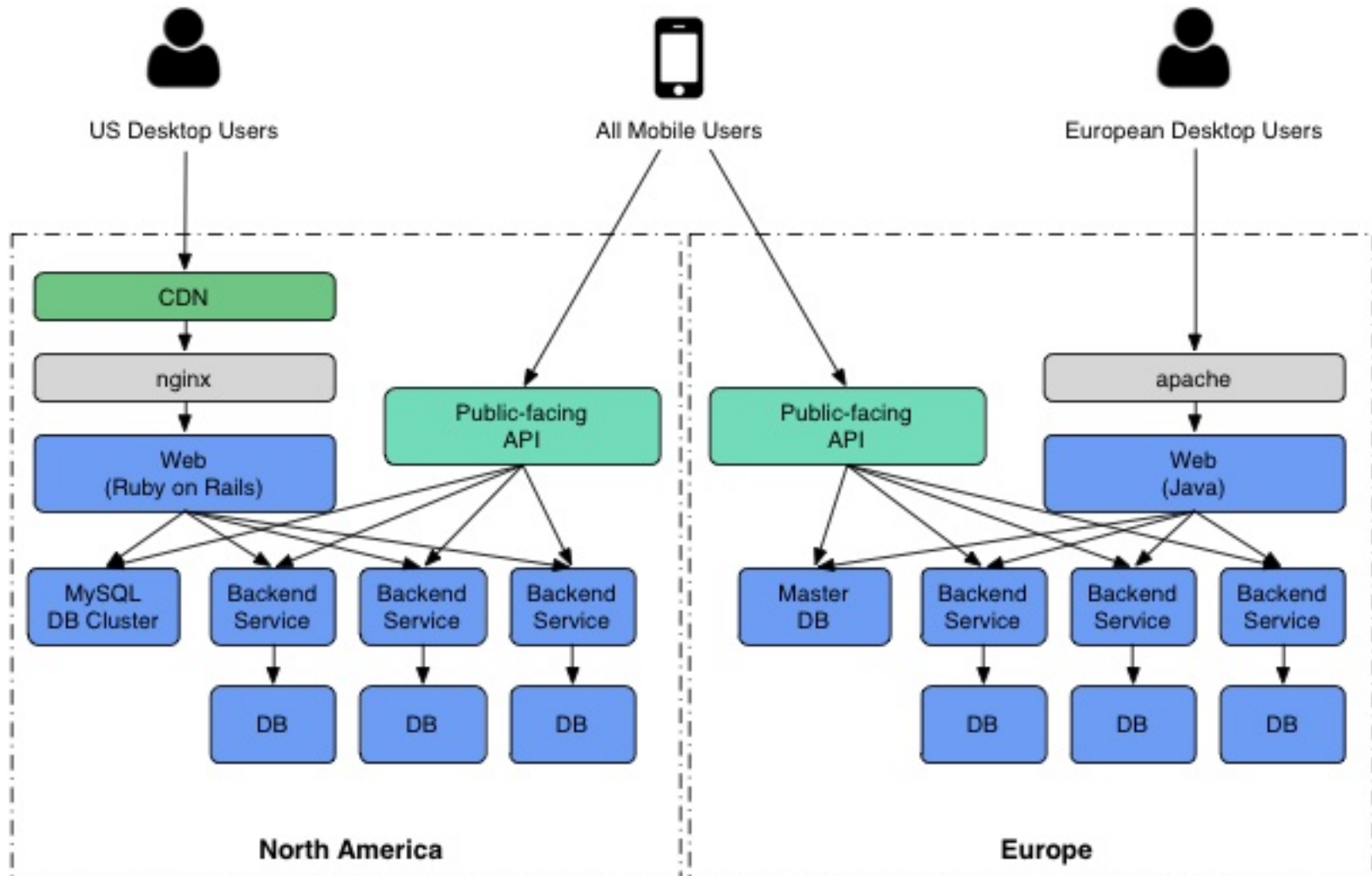


ensuring performance
scalability with CDN
(Content Distribution Network)
and data access services



heterogeneous architecture

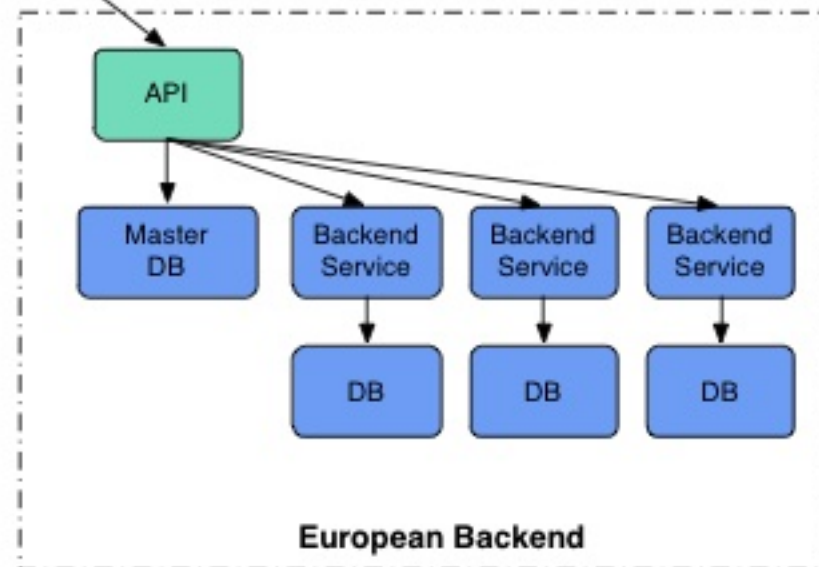
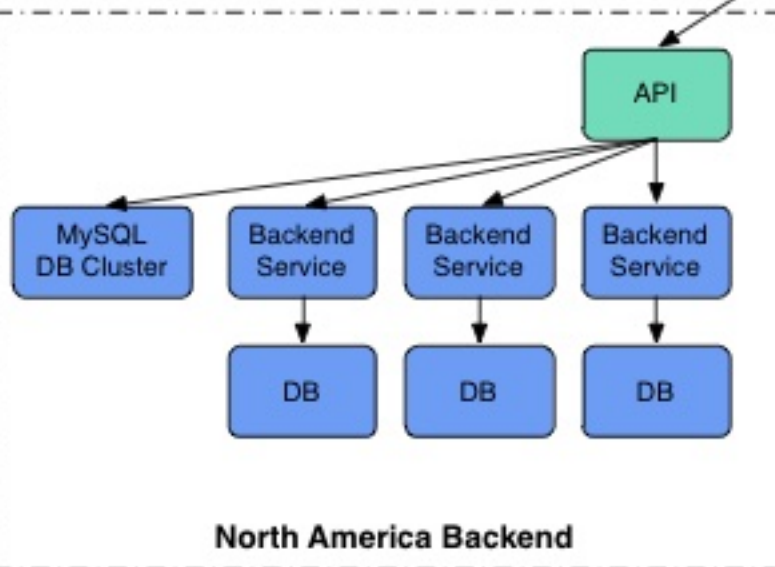
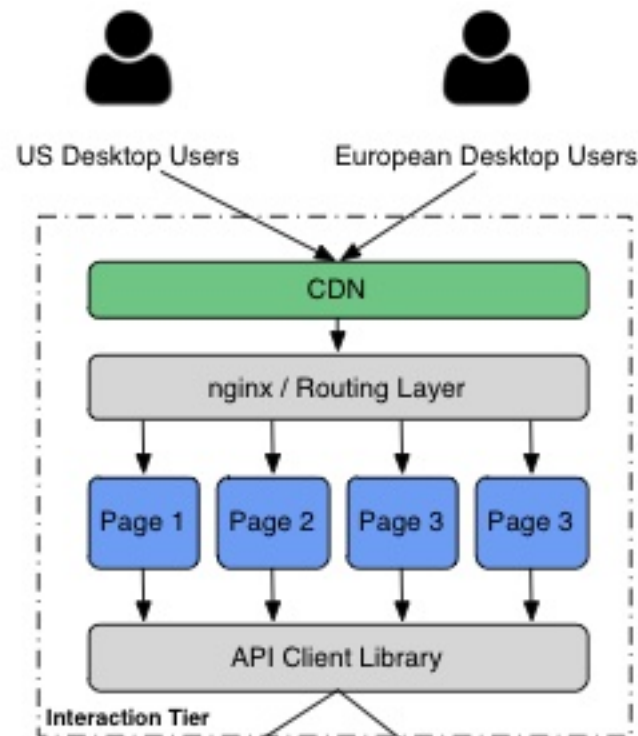
distinct implementations depending on the geographical area



providing support
for interactions with mobile devices
dedicated API, access depending on user location

advanced

redesign
(~20 distinct,
independent
Web
applications





Can we use Web services (APIs) for authorization and authentication?



authorization

Essential steps:

obtaining the access key 

application authentication + authorization 

obtaining user consent   

calling the functionalities of the service (via API) 

authorization

Usual steps to implement an application that will invoke a Web service based on a public API:

(1) registration of the implemented application via the site of the service provider

access key 

API key, consumer key, developer key

authorization

Usual steps to implement an application that will invoke a Web service based on a public API:

(1) registration of the implemented application via the site of the service provider

access key 

API key, consumer key, developer key

What Is an API Key? How API Keys Work (with Examples)
api7.ai/blog/what-is-api-key-why-need-them-complete-guide

authorization

Usual steps to implement an application that will invoke a Web service based on a public API:

(2) based on this key, the application will be able to authenticate in order to be authorized to access the desired service

various access policies may be imposed (permissions):



consultation only (read), editing possibility, etc.

authorization

Usual steps to implement an application that will invoke a Web service based on a public API:

(3) application authentication and authorization takes place with the consent of the user

if the user is not authenticated, the login information will be requested (e.g., name + password)   
– eventually, using 2FA (Two Factor Auth) –,
then he/she will be able to authorize the application to access the data via the provided Web service

authorization

Usual steps to implement an application that will invoke a Web service based on a public API:

(4) the application calls the functionalities offered by the service to receive/modify data of interest, according to the access policies

the current session will be established and maintained based on authentication information
– usually, stored and transported via tokens (auth tokens)

oauth

Authorizing an application to access private data in a standardized way – based on current Web technologies – can be done via OAuth

oauth

Authorizing an application to access private data in a standardized way – based on current Web technologies – can be done via **OAuth**

open protocol – RFC 6749

OAuth 1.0 (2010), **OAuth 2.0** (2012), OAuth 2.1 (draft)

oauth.net/2/

advanced



Authorize GraphQL API Explorer



GraphQL API Explorer by [github](#)
wants to access your [busaco](#) account



Discussions
Read access to Team discussions.



GPG Keys
Read access to your GPG keys.



Organizations and teams
Read-only access



Packages
Read packages from GitHub Packages. Delete private packages from GitHub Packages.



Public SSH keys
Read-only access



Repository webhooks and services
Read-only access



Repositories
Public and private

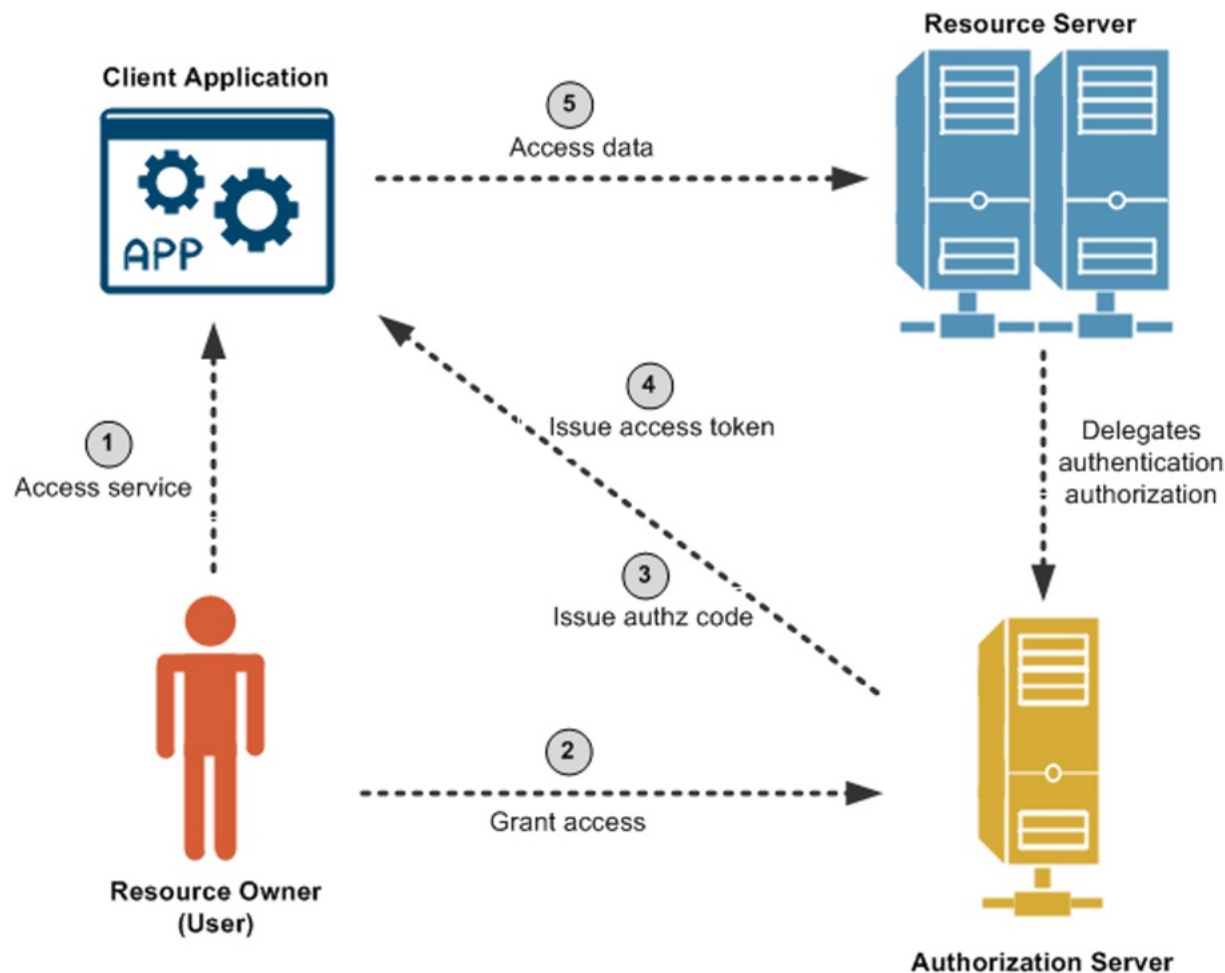


Personal user data
Full access



authorizing through **OAuth**
the Web application
GraphQL API Explorer
to obtain access to data of the
user [busaco](#)
owned by **GitHub**

advanced



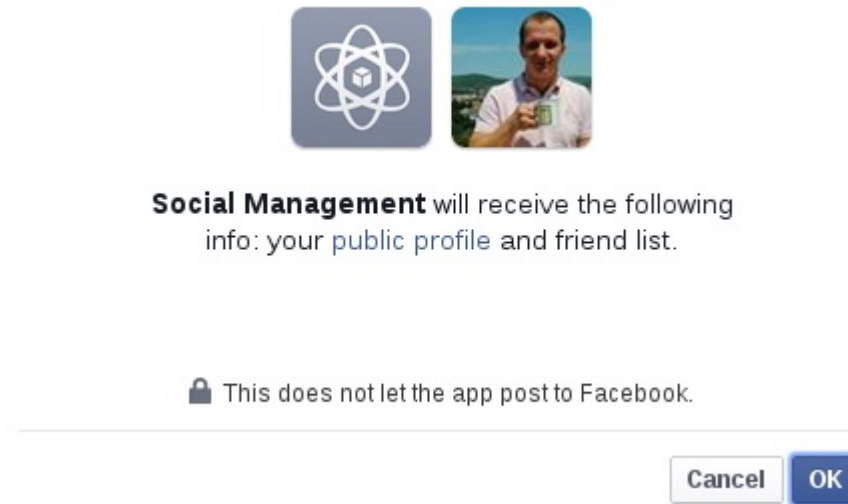
processes for
authorizing
access to a Web
resource

visual guides:

dev.to/jaypmedia/oauth-20-explained-in-simple-terms-with-visuals-30he

bytebytego.com/guides/oauth-20-flows/

medium.com/codenx/oauth-2-0-4cddd6c7471f



example – Facebook:

authorization with various permissions – e.g., age_range,
email (access to a user's e-mail address),
public_profile, user_birthday, user_hometown, user_friends,
user_likes, user_photos, rsvp_event and others
developers.facebook.com/docs/permissions

oauth

Libraries (server and/or client) available for
C, C++, Go, Erlang, Java, JavaScript, Objective-C, .NET,
Perl, PHP, Python, Ruby, Rust, Swift,...

oauth.net/code/

for the Web browser:
[helloJS](#), [JSO](#), [SalteAuth](#)

oauth

Proxy services for authentication/authorization through OAuth

Auth0 – auth0.com/docs/

Ory Hydra – www.ory.sh/hydra/

OAuth.io – github.com/oauth-io

Okta – developer.okta.com

SimpleLogin – simplelogin.io/developer/

SSQ signon – github.com/ssqsignon

oauth

Authorization via specific Web services – examples:

GitHub

docs.github.com/en/apps/oauth-apps/building-oauth-apps

Google

developers.google.com/identity/protocols/oauth2

Stack Exchange

api.stackexchange.com/docs/authentication

WordPress

developer.wordpress.com/docs/oauth2/

Step 1 Select & authorize APIs

Select the **scope** for the APIs you would like to access or input your own OAuth scopes below. Then click the **"Authorize APIs"** button.

▶  Contact Center AI Platform API v1alpha1

▶  Contacts v3

<https://www.google.com/m8/feeds/>

▶  Container Analysis API v1

▶  Content API for Shopping v2.1

▶  Data Labeling API v1beta1

▶  Data Lineage API v1

▶  Data Portability API v1

▶  Data pipelines API v1

▶  Database Migration API v1

Input your own scopes

Authorize APIs

Step 2 Exchange authorization code for tokens

Step 3 Configure request to API

Request / Response

advanced

```
client_secret=...&grant_type=refresh_token&refresh_token=1%2F%2F04MYVWnB6dWOXCgYIARAAGANwF-L9IrjmuJmZ8ppjjchrXUMGYnCXpZYeJ-xIfMrPNYJ7S0nUg-QjS9brBFj1mEQxXZcTCGkMo&client_id=407408718192.apps.googleusercontent.com
```

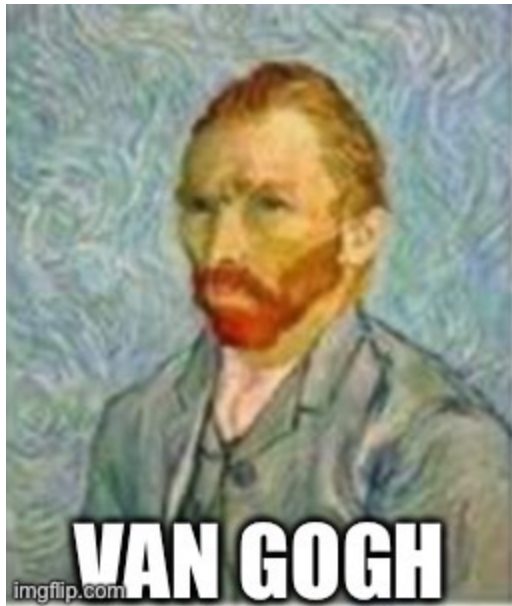
```
HTTP/1.1 200 OK
Content-length: 387
X-xss-protection: 0
X-content-type-options: nosniff
Transfer-encoding: chunked
Expires: Mon, 01 Jan 1990 00:00:00 GMT
Vary: Origin, X-Origin, Referer
Server: scaffolding on HTTPServer2
-content-encoding: gzip
Pragma: no-cache
Cache-control: no-cache, no-store, max-age=0, must-revalidate
Date: Fri, 16 May 2025 13:12:08 GMT
X-frame-options: SAMEORIGIN
Alt-svc: h3=":443"; ma=2592000,h3-29=":443"; ma=2592000
Content-type: application/json; charset=utf-8
```

```
{
  "access_token": "ya29.a0AW4Xtxji9ru3u3CXro7swZm-MMehRoi7z9J8l_KE263ZFjAypbK-rx9ZjQdMElT4LMS9UrYy-60WDBS3BcWtYgAXCACRCbg8-NVLEDliqXeUQC_yYE4BMRFbbkECQaGNQhDyD3e43GKud1bGZ0vWeFPJVZQXu895wt4aCgYKAfASARASFQHGx2Mia3_SUGmGdY8LvOiiQu2mA0175",
  "scope": "https://www.googleapis.com/auth/contacts",
  "expires_in": 3599,
  "refresh_token_expires_in": 604137,
  "token_type": "Bearer"
}
```

the current session is kept in a JSON token

experiment access to various data (e.g., contacts via Google Contacts API) based on **OAuth 2.0 Playground**
developers.google.com/oauthplayground/

(instead of) break



authentication

Authentication methods:

based on the Web session via SID (Session Identifier)

implicitly, the Web is stateless

authentication

Authentication methods:
using tokens

token based authentication

authentication

Authentication methods:
without a password (passwordless)

one-time-use URL

example: **Tumblr**

guide: PHP: T. Boronczyk, *Generating One-Time Use URLs* (2024)

www.sitepoint.com/generating-one-time-use-urls/

also study TOTP (Time-Based One-Time Password Algorithm)

www.twilio.com/docs/glossary/totp • RFC 6238: tools.ietf.org/html/rfc6238

authentication

Authentication methods:

SSO (Single Sign-On)

authenticating users in several related applications

authentication

Authentication methods:

social authentication via other user accounts

for example, via social networks
or other trusted websites

authentication

Authentication methods:

2FA (Two-Factor Authentication)

the user must provide at least 2 factors
regarding his/hers identity:

knowledge or objects held exclusively by that person

brainstation.io/cybersecurity/two-factor-auth

authentication

Authentication methods:

using a passkey

multi-factor authentication mode:

something you **know**: a PIN used to unlock the device

something you **have**: the authenticator

(a security key or something embedded in a personal device)

something you **are**: a fingerprint, a scan of user's face

passkey.org

fidoalliance.org/passkeys/

authentication

Authentication methods:

biometric authentication

fingerprint recognition

facial recognition or other organs

eye scan (iris, retina)

voice identification

genetic code analysis (DNA matching)

authentication

Authentication methods:

authentication via hardware device

typical examples:

smartcard

personal device (e.g., watch) via wireless interactions

authentication

Authentication methods – overview:

- based on Web session
- using tokens (token based authentication)
- without password (passwordless)
- SSO (Single Sign-On)
- social authentication
- 2FA (Two-Factor Authentication)
- passkeys
- biometric authentication
- authentication via hardware device

details in *The Copenhagen Book* (2026): thecopenhagenbook.com

authentication

In the context of Web applications, also consider
*Web Authentication: An API for accessing Public Key
Credentials – Level 2 (W3C Recommendation, 2021)*

www.w3.org/TR/webauthn-2/

user guide:

webauthn.guide

demo code:

webauthn.io

specification level 3

draft (6 May 2026): w3c.github.io/webauthn/

authentication

Observation:

there are cases where a Web application does not require user management and authentication

real examples:

Awesome (free) Web apps that work without login
github.com/aviaryan/awesome-no-login-web-apps

openid

OpenID

decentralized way of user authentication at the Web level
based on the SSO paradigm

the user can prove that it owns a specific URL meant to
identify him/her online via a provider (service) of
digital identity (identity provider)
e.g., using a social Web application

openid

Each identify of a user is designated by an URL
(established by the digital identity provider)

example: steamcommunity.com/openid/id/steamid

to confirm the identity,
the user will have to authenticate

openid

OpenID Connect

Offers an identity layer based on the OAuth 2 protocol

used data format: JWT – JSON Web Token
standardized by RFC 7519

openid.net/developers/how-connect-works/

A **JWT** token includes a JSON object used for “secure” data exchange between two entities – usually, a client invoking a Web service (API)

A **JWT** token includes a JSON object used for “secure” data exchange between two entities – usually, a client invoking a Web service (API)

real cases:

authentication

user identification after successful authentication is carried out via a JWT token (ID token) whose structure is specified by the OpenID Connect specification

openid.net/specs/openid-connect-core-1_0.html#IDToken

A **JWT** token includes a JSON object used for “secure” data exchange between two entities – usually, a client invoking a Web service (API)

real cases:

authorization

each request from the authorized client to invoke a functionality of the Web service will include the corresponding JWT token (similar to Web sessions)

A **JWT** token includes a JSON object used for “secure” data exchange between two entities – usually, a client invoking a Web service (API)

real cases:

data exchange

the JWT tokens can be used to ensure the integrity of the data exchange (e.g., detection of unauthorized modification attempts of the information by third parties)

A **JWT** token includes a JSON object used for “secure” data exchange between two entities – usually, a client invoking a Web service (API)

the transmitted data can be verified and considered trustable via digital signatures

various digest algorithms are used to ensure the authenticity of the messages (HMAC – Hash-based Message Authentication Code) in conjunction with public/private cryptographic key pairs

JWT token

```
eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXVCJ9.eyJzdWIiOiJzdHVkZW50NjkiLCJuYW11IjoiaVHV4eSBQaW5ndWlubmVzY29vbCI6ImFkbWluIjpmYXxzZSwiaWF0IjoxNzc5MDA5MDIwMDAwfQ.cRItHxvEzi4tH9op4oBVICm1nJMIVtLKNKQNWbMSliE
```

messages – of authentication
or exchange of authorization
information (credentials) –
transmitted in **JWT** format
based on a cryptographic
algorithm

The image shows the JWT.io token decoder interface. It displays a JWT token and its decoded components. The token is: `eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXVCJ9.eyJzdWIiOiJzdHVkZW50NjkiLCJuYW11IjoiaVHV4eSBQaW5ndWlubmVzY29vbCI6ImFkbWluIjpmYXxzZSwiaWF0IjoxNzc5MDA5MDIwMDAwfQ.cRItHxvEzi4tH9op4oBVICm1nJMIVtLKNKQNWbMSliE`

Header

> Algorithm & Token Type

```
{
  "alg": "HS256",
  "typ": "JWT"
}
```

✓ Valid header

Payload

> Data

```
{
  "sub": "student69",
  "name": "Tuxy Pinguinnescool",
  "admin": false,
  "iat": 1779009020000
}
```

✓ Valid payload

Sign JWT BASE64URL ENCODED ☐

> Secret

A key of 256 bits or larger MUST be used with HS256 as specified on RFC 7518.

resources + instruments: jwt.io

JWT token

```
eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXVCJ9.eyJzdWIiOiJzdHVkZW50NjkiLCJuYW
1lIjoivHV4eSBQaW5ndWlubmVzY29vbCI6ImFkbWluIjpmYXZzZWiaWF0IjoxNzc5M
DA5MDIwMDAwfQ.cRIthxvEzi4tH9op4oBVICm1nJMIVtLKNKQNwbMSliE
```

data stored by a JWT token
resulting after decoding

security attacks on JWT: portswigger.net/web-security/jwt

Decoded Header

JSON Claims Breakdown

alg	HS256	The algorithm used to sign the JWT. Learn more
typ	JWT	The media type of this complete JWT. Learn more

Decoded Payload

JSON Claims Breakdown

sub	student69	The subject of the JWT (the user). Learn more
name	Tuxy Pinguinnescool	
admin	false	
iat	1779009020000 (Fri Jul 21 58344 07:13:20 GMT+0300 (Eastern European Summer Time))	The time at which the JWT was issued. Learn more

This value must be a [NumericDate](#) type, representing seconds.

JWT Signature Verification (Optional) BASE64URL ENCODED ☐

Enter the secret used to sign the JWT below:

>_ Secret

A key of 256 bits or larger MUST be used with HS256 as specified on RFC 7518.

🔗 Alternatives to REST?



data access model: **graphql**

Graph Query Language

“a query language for APIs and a runtime for fulfilling those queries with your existing data”

graphql.org

under the jurisdiction of the Linux Foundation

gql.foundation

data access model: graphql

Graph Query Language

declarative

inspired by JSON

strict (strong-typed)

data access model: graphql

Graph Query Language

queries (objects of type **Query**), designating read operations, and **updates** (**mutations** – objects of type **Mutation**), that are data alteration operations, are allowed

graphql.org/learn/queries/

data access model: graphql

Graph Query Language

support is provided for anticipating
what data will be returned + its structure
(types + schemas)

graphql.org/learn/schema/

data access model: **graphql**

Graph Query Language

the relations between different (categories of) resources
can be specified via links – connections

data is modeled as graphs – hypermedia resources
graphql.org/learn/thinking-in-graphs/

data access model: graphql

Graph Query Language

the returned answer includes only
the data that has been requested
-> improving client-level performance

solving the problems regarding over/under fetching
(getting more or less data)

nordicapis.com/is-graphql-the-end-of-rest-style-apis/
hasura.io/blog/rest-view-of-graphql/
philsturgeon.com/graphql-vs-rest-overview/

	GraphQL	REST
entity	resource	resource
format	JSON	any Media Type (MIME) usually: JSON
protocol	HTTP – usually, adopts its own conventions	protocol independent (usually, HTTP)
who decides what data will be returned	the client	the server
endpoints	a single endpoint to provide related data, if relationships have been specified between them	multiple endpoints (independent)
data types	strong (explicitly declared types)	weak (type checking is not mandatory)
client-server relation	fat client—fat server	thin client—fat server
documentation	self-describing	needs third party solutions (e.g., OpenAPI Specification)
vision	query language, specification, collection of tools	architectural style

GraphQL as an alternative to developing services via the REST paradigm

data access model: graphql

Aspects to be considered by implementers:

security problems

– e.g., authentication, authorization, denial of service

support for caching must be offered explicitly

version management

blog.pusher.com/rest-versus-graphql/

aws.amazon.com/compare/the-difference-between-graphql-and-rest/

data access model: graphql

Tools for developers:

reference implementation for the server (Node.js):

GraphQL.js

graphql.org/graphql-js/

www.apollographql.com/blog/how-to-use-graphql-with-javascript-graphql-js-tutorial/

libraries available for C, Go, Java, Kotlin,
.NET, PHP, Python, Swift, etc. – graphql.org/code/

data access model: graphql

Tools for developers:

interactive Web editors

Altair GraphQL Client – altairgraphql.dev

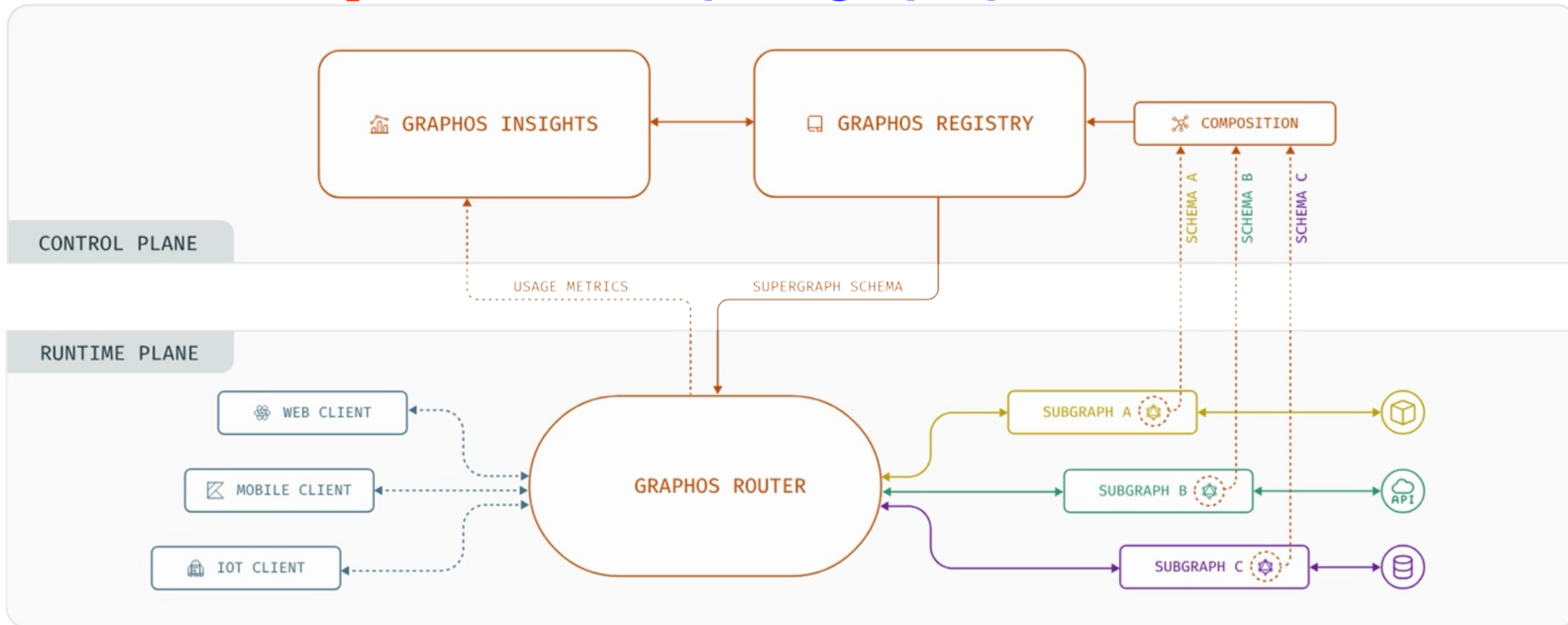
GraphiQL – github.com/graphql/graphiql

GraphQL Editor – graphqleditor.com

data access model: graphql

Tools for developers:

Apollo – www.apollographql.com/docs/



data access model: graphql

Tools for developers:

GraphQL Yoga 2 – the-guild.dev/graphql/yoga-server/docs

GraphQL server based on the GraphQL over HTTP spec
(e.g., suport pentru Fetch API, Server-Sent Events)

data access model: graphql

Other servers + frameworks:

API Platform Framework (PHP) – api-platform.com

graphql-php (PHP) – webonyx.github.io/graphql-php/

Graphene (Python) – graphql-python.org

GraphQL Java – www.graphql-java.com

GraphQL Kotlin – github.com/ExpediaGroup/graphql-kotlin

GraphQL for .NET – github.com/graphql-dotnet/

gqlgen (Go) – gqlgen.com

Hot Chocolate (.NET) – chillicream.com/docs/hotchocolate

GraphiQL

Prettify

History

Explorer

advanced

1 query {

2 viewer {

3 name

4 createdAt

5 repositories (first: 1) {

6 nodes {

7 description

8 primaryLanguage {

9 name

10 }

11 }

12 }

13 }

14 }

query

Language.name: String!

The name of the current language.

{

"data": {

"viewer": {

"name": "Sabin Buraga",

"createdAt": "2015-04-03T09:57:23Z",

"repositories": {

"nodes": [

{

"description": "Demo App for the DBMSs Ontology

"primaryLanguage": {

"name": "JavaScript"

}

}

]

}

}

}

}

results

interactive testing with the **GraphiQL** instrument
of the GitHub API via GraphQL: docs.github.com/en/graphql

< User	RepositoryOwner
Represents an owner of a Repository.	/ schema
FIELDS	
avatarUrl(size: Int): URI! A URL pointing to the owner's public avatar.	
id: ID!	
login: String! The username used to login.	
repositories(privacy: RepositoryPrivacy orderBy: RepositoryOrder affiliations: [RepositoryAffiliation] ownerAffiliations: [RepositoryAffiliation] = [OWNER, COLLABORATOR] isLocked: Boolean after: String before: String first: Int last: Int isFork: Boolean): RepositoryConnection! A list of repositories that the user owns.	

advanced

Concepts defined by data schemas that can use scalar types (Boolean, Int, String,...) or specified by the programmer – in this case, for example URI, User, RepositoryOwner

[] specifies arrays of objects

! indicates non-nullable data (non-zero values will be given always)

```

query {
  viewer {
    name
  }
}

```

1: GraphQL request made by client

name
__typename
twitterUsername
sponsorshipsAsMaintainer
organizationVerifiedDomainEmails
String The user's public profile name.

advanced

▼ POST

Scheme: https

Host: graphql.github.com

Filename: /graphql/proxy

Address: 34.203.109.182:443

2: sending the request through POST

Status 200 OK ?

Version HTTP/1.1

Transferred 1.53 kB (78 B size)

Status	Method	Domain	File	Initiator	Type	Transferred	Size
200	POST	graphql.github.com	proxy	graphql-explorer-f2ef65f56c75783cdd28.js:2 (fetch)	json	1.53 kB	78 B

```

{ "query": "query { \n viewer { \n  name\n  createdAt\n } \n}",
  "variables": {}
}

```

3: JSON data sent to the server

```

{ "data": {
  "viewer": {
    "name": "Sabin Buraga",
    "createdAt": "2015-04-03T09:57:23Z"
  }
}

```

4: the answer provided in JSON format

```
{
  "errors": [
    {
      "path": [ ... ],
      "extensions": {
        "code": "variableNotDefined", "variableName": "num"
      },
      "locations": [ {
        "line": 5,
        "column": 26
      } ],
      "message": "Variable $num is used by anonymous query
but not declared"
    }
  ]
}
```

possible way to report errors/exceptions

1 ▾ {
2 ▾ country(code: "RO") {
3 capital
4 currency
5 emoji
6 ▾ languages {
7 name
8 native
9 }
10 ▾ continent {
11 name
12 }
13 }
14 }

▶

🔍

🗑️

📄

name String!

query

+ countries.trevorblades.com

▾ {
▾ "data": {
▾ "country": {
"capital": "Bucharest",
"currency": "RON",
"emoji": "🇷🇴",
"languages": [
{
"name": "Romanian",
"native": "Română"
},
],
"continent": {
"name": "Europe"
}
}
}
}
}

results

MISS 267ms

information about countries
provided by a GraphQL API:
countries.trevorblades.com

country(
code: ID!
): Country

schema

TYPE DETAILS

type Country {
code: ID!
name: String!
native: String!
phone: String!
continent: Continent!
capital: String
currency: String
languages: [Language!]!
emoji: String!
emojiU: String!
states: [State!]!
}



CHANGE SCHEMA

< Business

Query

Search Query...

No Description

FIELDS

business(

id: String,

locale: String

): Business

reviews(business, locale): Reviews

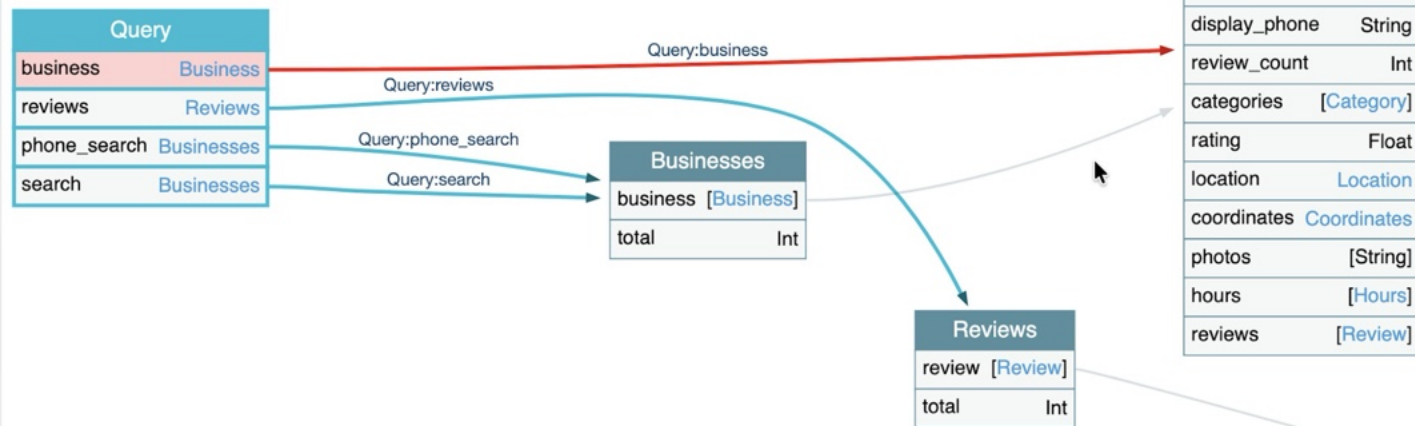
phone_search(phone): Businesses

search(term, location, country, offset, limit,
sort_by, locale, longitude, latitude,
categories, open_now, open_at, price,
attributes, radius): Businesses

exploring a GraphQL API

– here, Yelp API – using the Web tool

GraphQL Voyager



Query



Sort by Alphabet



Skip Relay



Skip deprecated



Show leaf fields

Status	Method	File	Initiator	Size
304	GET	tags.json	main-8a26b4a660344706.js:1 (fetch)	24.22 kB
200	GET	i?stm=1741672572539&e=pp&url=	_app-ffe7ac3dcba3a84d.js:1 (fetch)	43 B
200	GET	i?stm=1741672582544&e=pp&url=	_app-ffe7ac3dcba3a84d.js:1 (fetch)	43 B
200	POST	graphql?consumer_key=94110	_app-ffe7ac3dcba3a84d.js:1 (fetch)	41 B
200	GET	i?stm=1741672583644&e=ue&eid=	_app-ffe7ac3dcba3a84d.js:1 (fetch)	43 B

advanced

Headers

Cookies

Request

Response

Filter Request Parameters

JSON

Raw

```

operationName: "ItemDelete"
query: "\n mutation ItemDelete($itemId: ID!) {\n
      deleteSavedItem(id: $itemId)\n
    }\n"
variables: {
  itemId: "3148910115"
}

```

real case:

Pocket – a delete operation of a bookmark via GraphQL

graphql: resources + case studies

The Fullstack Tutorial for GraphQL

www.howtographql.com

Awesome GraphQL

github.com/chentsulin/awesome-graphql

GraphQL APIs

github.com/APIs-guru/graphql-apis

Web – key ingredient of cloud technologies

cloud computing



software

forecasts
(finance,
transport,...)

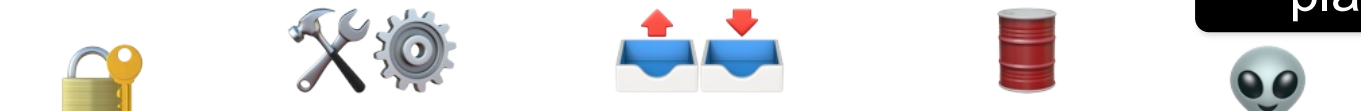
e-commerce

news
interpersonal
Communic.

monitoring

tourism

amusement



platform

security

software
execution

processing
message queues

data-
bases

identity



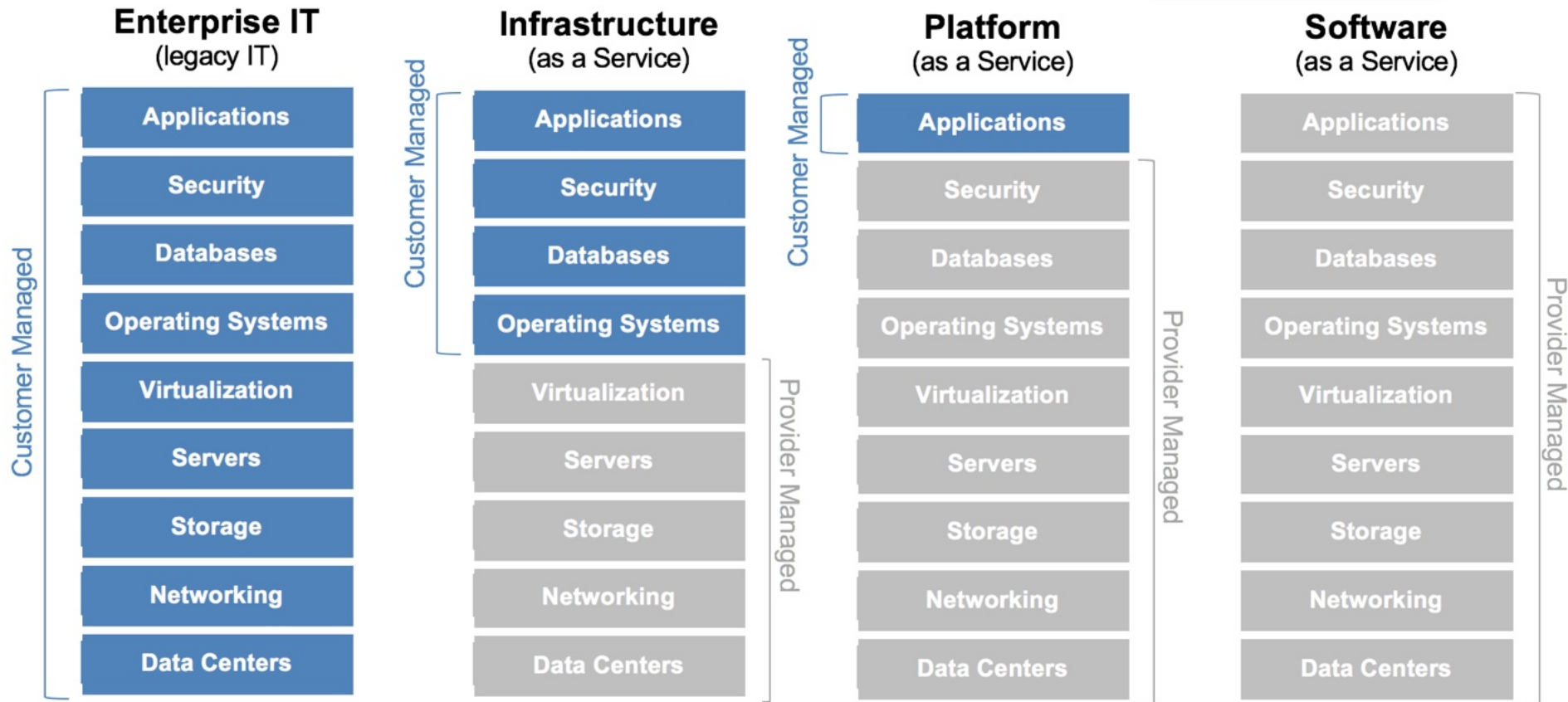
infrastructure

storage

compute

network





Some functionalities can be managed on-premises or by a provider of services available in cloud – according to (Eizadirad, 2017)

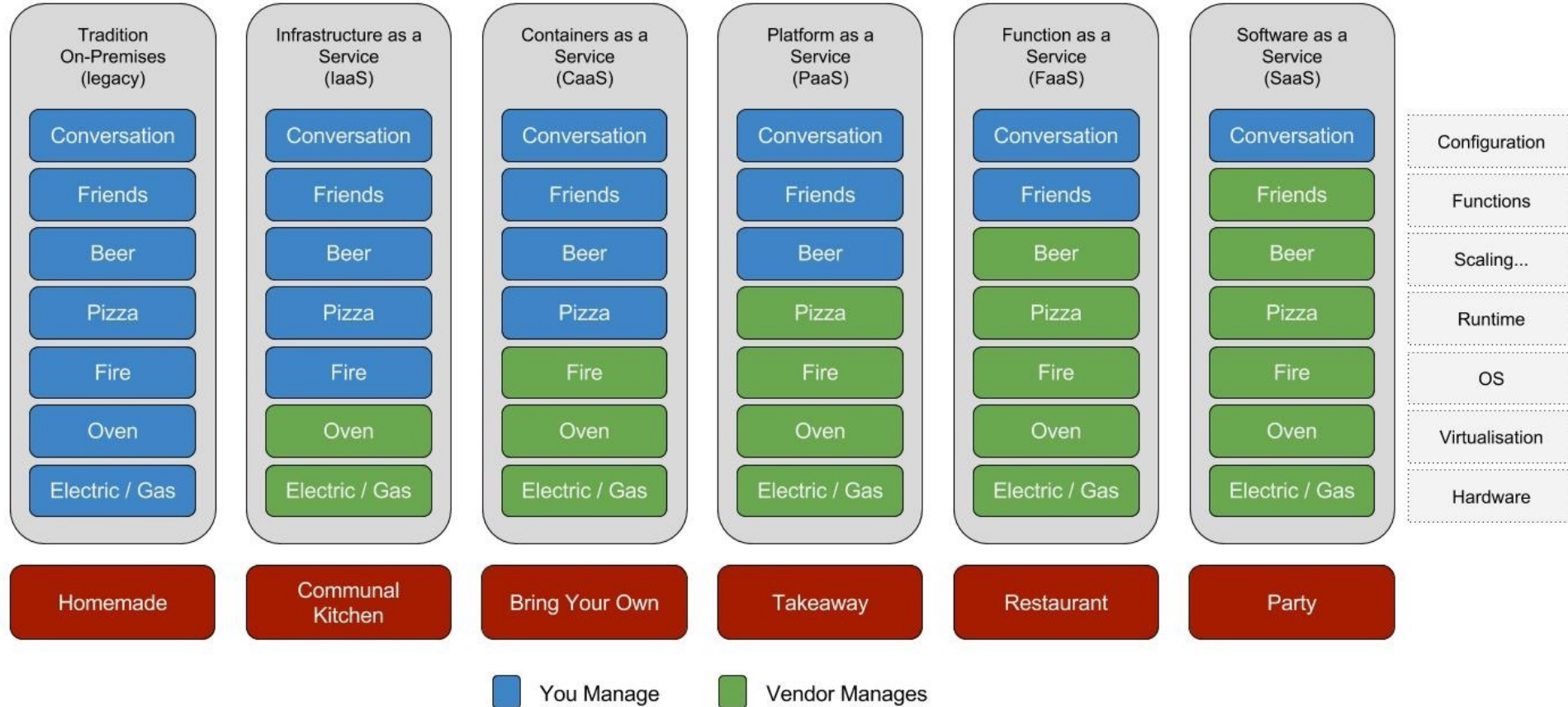
www.linkedin.com/pulse/iaas-paas-saas-explained-compared-arsalan-eizadirad



Pizza as a Service 2.0

<http://www.paulkerrison.co.uk>

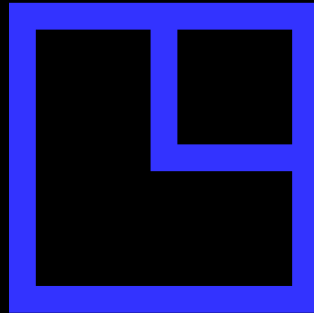
advanced



P. Kerrison, Pizza As A Service 2.0 (2017)
www.paulkerrison.co.uk/random/pizza-as-a-service-2-0

summary

Web services development



from microservices and API specification
to serverless and data access via GraphQL

next episode: other Web application architectures

